

Behemoth: transparent polynomial commitment scheme with constant opening proof size and verifier time

István András Seres and Péter Burcsi

Eötvös Loránd University

Abstract. Polynomial commitment schemes are fundamental building blocks in numerous cryptographic protocols, e.g., verifiable secret sharing, zero-knowledge succinct non-interactive arguments, and many more. The most efficient polynomial commitment schemes rely on a trusted setup, which is undesirable in trust-minimized applications, e.g., cryptocurrencies. However, transparent polynomial commitment schemes are inefficient (polylogarithmic opening proofs and/or verification time) compared to their trusted counterparts. It has been an open problem to devise a transparent, succinct polynomial commitment scheme or prove an impossibility result in the transparent setting. In this work, for the first time, we create a transparent, constant-size polynomial commitment scheme called Behemoth with constant-size opening proofs and a constant-time verifier. The downside of Behemoth is that it employs a cubic prover in the degree of the committed polynomial. We prove the security of our scheme in the generic group model and discuss parameter settings in which it remains practical even for the prover.

Keywords: Polynomial commitment schemes · groups of unknown order · KZG

1 Introduction

A polynomial commitment scheme (PCS or **PC**) allows a prover to commit to a polynomial f of degree maximum d (typically over a finite field, i.e., $f \in \mathbb{F}_p^{\leq d}[x]$, and $p \approx 2^{256}$, $d \approx 2^{10} - 2^{30}$). Importantly, the prover can later open the committed polynomial at any point. Specifically, the prover can convince a verifier about evaluations of f at point $z \in \mathbb{F}_p$ of the polynomial f when the verifier is given only a short commitment to f and the statement $f(z) = s$. PCSs were first proposed by Kate, Zaverucha, and Goldberg (KZG) [37]. Since then, PCSs have become the cornerstone of many important cryptographic protocols, such as verifiable secret sharing, zero-knowledge sets, zero-knowledge succinct non-interactive arguments of knowledge (zkSNARKs) [12], Verkle trees [40] and many more. Therefore, it is of great interest to improve existing PCSs, as they directly translate to efficiency and trust assumption improvements in numerous applications. The most efficient PCSs are derived from the original KZG scheme [17, 25, 31]. The KZG scheme and its extensions offer short commitments, evaluation proofs (both one group element), and batching capabilities for the opening protocol at the expense of a *trusted setup* and a linear-sized (in the maximum degree d of the committed polynomials) common reference string.

A trusted setup requires private randomness to generate the public parameters necessary for the commitment scheme. The private randomness (“toxic waste”) must be kept

secret or discarded to maintain the soundness of the PCS. If the randomness used in the setup is known to an adversary, it is possible to break the binding property of the PCS or create invalid opening proofs. Therefore, a successful trusted setup is paramount for the security of the KZG PCS and its variants. Trusted setups are straightforward to perform if there exists a trusted third party who does not disclose private randomness. In the absence of a trusted party to distribute the trust in trusted setups, numerous multi-party protocols have been suggested for executing the KZG trusted setup [11, 21, 28, 39, 46]. However, trusted setups are usually undesirable in trust-minimized and decentralized applications, such as cryptocurrencies. To that end, significant efforts have been dedicated to devising transparent PCSs whose setup algorithm uses only public coins.

There is a multitude of transparent PCSs. They use various techniques and are instantiated under diverse cryptographic assumptions. FRI [10] and its variants [3, 38] only assume the existence of one-way functions, hence post-quantum secure. Bootle et al. [19, 57] and Bünz et al. [22] assume the discrete logarithm assumption in cyclic groups. Lee builds a transparent **PC** in groups with non-degenerate, efficiently computable, bilinear pairings [43]. A recent line of research designs PCSs [4, 13, 23, 26] in groups of unknown order (GUO). An advantage of applying groups of unknown order in cryptographic applications is that some families of GUOs can be instantiated transparently [29]. Specifically, class groups of imaginary quadratic fields and Jacobians of hyperelliptic curves [56] can be sampled efficiently and transparently using only public randomness. For efficiency reasons, we focus mainly on instantiating our scheme in class groups. Currently, the best known algorithms can compute the order of an imaginary class group in subexponential time [35, 53]. So far, all transparent **PC** schemes have had polylogarithmic evaluation proofs and/or verifiers.

Ideally, one wants to match the efficiency properties of the KZG **PC** scheme also in the transparent setting, i.e., both constant opening proofs and verifiers with possibly batching capabilities. Hence, a natural question arises:

Is there a transparent, succinct PCS that achieves constant-size opening proofs and constant-time verifiers with possibly batching capabilities?

In this work, we answer affirmatively. To our knowledge, we devise the first transparent PCS with *constant-size opening proofs and constant-time verifier*, i.e., the first transparent succinct polynomial commitment scheme.

Our contributions. In this work, we make the following contributions.

We propose Behemoth,¹ the first succinct (constant opening proof size and verifier time), transparent polynomial commitment scheme. We prove its security in the random oracle and generic group models. Thus, we positively answer an open question by Nikolaenko et al. [46] about the existence of succinct, transparent polynomial commitment schemes.

As an application of our transparent **PC** scheme, we prove the existence of a transparent zkSNARK with constant proof size and verifier time, cf. Section 6. To our knowledge, this is the first such construction in the polynomial-IOP paradigm.

¹ Behemoth is an enormous biblical monster described in the Book of Job. The name of the scheme alludes to the huge integers used in our scheme.

The rest of this paper is organized as follows. We provide a technical overview in Section 2. In Section 3, we introduce the pertinent background on polynomial commitment schemes, computational hardness assumptions in groups of unknown order, the generic group model, and the applied non-interactive zero-knowledge proofs. Section 4 describes the first succinct and transparent polynomial commitment scheme whose security is proven in Section 5. Using our polynomial commitment scheme, we prove the existence of constant, transparent SNARKs in Section 6. We evaluate the theoretical and practical performance of Behemoth in Section 7. Finally, we conclude our work in Section 8 by pointing out several remaining open problems and research directions.

2 Technical overview

This section presents the main ideas behind our polynomial commitment construction. Our end goal is to instantiate the KZG PCS in a group of unknown order and not to resort to techniques (e.g., inner product arguments or Merkle trees) that seem to inherently lead to (poly)logarithmic proofs and/or verifiers.

First, we recall the KZG polynomial commitment scheme. In the KZG scheme, the public parameters consist of the structured reference string produced by the trusted setup: $\text{srs} = \{g^{\tau^i}\}_{i=0}^d$ for $\tau \in_R \mathbb{F}_p^*$. The commitment to a polynomial $f \in \mathbb{F}_p^{\leq d}[x]$ is the evaluation of the polynomial at a random point τ not known even to the committer, i.e., $g^{f(\tau)} \in \mathbb{G}$, for some prime-order elliptic curve group $\mathbb{G}$ ($|\mathbb{G}| = p$) with generator g . Furthermore, in the KZG scheme, we assume that $\mathbb{G}$ is equipped with an efficiently computable, non-degenerate bilinear pairing, i.e., $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$. More interestingly, in the opening proof of KZG for the statement $f(z) = s$, given the commitment $g^{f(\tau)}$, the prover can convince the verifier by sending a commitment $g^{q(\tau)}$ to the quotient polynomial $q(x) := \frac{f(x)-s}{x-z}$. This is correct because the verifier can check the equality of polynomials $q(x)(x-z) = f(x) - s$ in the exponent at the random point τ using the structured reference string and the bilinear pairing. The verification of the evaluation proof is achieved by checking $e(g^{q(\tau)}, g^{\tau-z}) \stackrel{?}{=} e(g^{f(\tau)-s}, g)$.

How can we possibly mimic the evaluation proof strategy of the KZG scheme in a group of unknown order $\mathbb{G}$? First, we observe that a KZG-style commitment $g^{f(\alpha)} \in \mathbb{G}$ binds the prover to the value of $f(\alpha)$, thanks to the unknown group order. We will show that for a public and properly chosen α this also implies polynomial binding.² Therefore, essentially, we have the same commitments to polynomials as the KZG scheme, i.e., $g^{f(\alpha)}$ for a carefully chosen public $\alpha \in \mathbb{Z}$. Since we do not know the order of the group, we can only evaluate polynomials over the integers in the exponent. We denote the evaluation of a polynomial over the integers as $\widehat{f}(\alpha)$ to disambiguate from the “regular” evaluation $f(\alpha)$ over the finite field $\mathbb{F}_p$. We use integer representatives from $[0, p)$ to lift the polynomial. The first challenge is to ensure polynomial binding by selecting α carefully. The intuition is that α must be large compared to $\mathbb{F}_p$ and d , so that $\widehat{f}(\alpha)$ uniquely determines $f(\alpha)$: the coefficients can be recovered from the base α representation of $\widehat{f}(\alpha)$. If α is small, say $\alpha = 5$, then given a commitment $g^{127} = g^{\widehat{f}(\alpha)}$, the prover can open this commitment to numerous polynomials, for instance, $f_1 = 127, f_2 = 6x + 97$ in

² We will make this precise later in Section 5.

fact for any $f_2 = ax + b$ for any $(a, b) \in \mathbb{Z} \times \mathbb{Z} : 5a + b = 127$. We detail in Section 4.2, how to choose α properly to ensure polynomial and evaluation binding in $\mathbb{F}_p^{\leq d}[x]$.

A major challenge that a (very) large value of α poses is constructing correct opening proofs. In the evaluation proofs, since we do not rely on private evaluation points, the verifier can check the equality $\widehat{q(x)}(x - z) = \widehat{f(x)} - \widehat{s}$ in the exponent at the point α even without bilinear pairings. Specifically, given a commitment $g^{\widehat{q(\alpha)}}$ to $q(x)$, the verifier can compute $(g^{\widehat{q(\alpha)}})^{\alpha - z}$ without a bilinear pairing, since α is public. A downside of this approach is that now, the right-hand side of the verification equation, $\widehat{f(\alpha)} - \widehat{s} \in \mathbb{Z}$ is a large integer with $\approx d \log(p)$ bits. We describe in Section 4.3 the applied techniques that keep both our opening proofs and verifier constant, and the committer efficient. Moreover, along the way, we need to solve many technical challenges to preserve evaluation binding and the knowledge soundness properties of our proposed PCS. A major technical difficulty in proving knowledge soundness is to show that the protocols prevent adversarial provers from using integer polynomials that are not obtained by correctly lifting a modular polynomial. As mentioned above, one can only work over the integers in the exponent of a group of unknown order. This necessitates “projecting” statements over the integers back to finite fields. We achieve this by applying several non-interactive zero-knowledge proofs, cf. Section 3.5, to ensure the polynomial/evaluation binding and knowledge soundness of our PCS.

3 Background

3.1 Notations

In the following, we will use multiplicative notation to denote the group operation in the applied groups $\mathbb{G}$ (of unknown order). To sample x from a set S uniformly at random, we write $x \in_R S$. Some protocols need to sample random integers from the set of the first 2^λ primes that are denoted as $\text{Primes}(\lambda)$, where λ is the security parameter. Let p denote a large odd prime. For an univariate polynomial $f(x) \in \mathbb{F}_p^{\leq d}[x]$ where $f(z) = s$, let $\widehat{f(z)} := \widehat{s}$ denote the evaluation of f at z *over the integers*. We use a Python-like notation to index lists and arrays, i.e., we refer to the i th element of a list l as $l[i]$. Let

$$\mathcal{B} := \max_{x \in \mathbb{F}_p, f(x) \in \mathbb{F}_p^{\leq d}[x]} \widehat{f(x)} = \sum_{i=1}^{d+1} (p-1)^i = (p-1) \frac{1 - (p-1)^{d+1}}{1 - (p-1)}. \quad (1)$$

$\mathcal{B}$ is a universal upper bound for the evaluation *over the integers* of polynomials from the polynomial ring $\mathbb{F}_p^{\leq d}[x]$. Let $v_p(x)$ be the p -adic valuation of x , i.e., the (possibly negative) exponent of p in the factorization of x . The commitment of a polynomial f is denoted as $\boxed{f}$. The prover and the verifier are denoted as $\mathcal{P}$ and $\mathcal{V}$, respectively.

3.2 Polynomial Commitment Schemes

A polynomial commitment scheme $\mathbf{PC} = (\text{GenSRS}, \text{Com}, \text{ComVerify}, \text{Open}, \text{OpenVerify})$ consists of five algorithms and allows to commit to a polynomial f and later “open” the commitment $\boxed{f}$ at point z by proving that for some value $s = f(z)$. More formally:

- GenSRS**($1^\lambda, d$): The key generation algorithm takes in a security parameter λ and a parameter d which determines the maximal degree of the committed polynomial. It outputs a structured reference string srs (the commitment key). Note that srs implicitly determines λ and d .
- Com**(srs, f, d): The commitment algorithm $\text{Com}(\text{srs}, f, d)$ takes in srs and a polynomial f with maximum degree d , and outputs a commitment $c(= \boxed{f})$ and a string $\text{hint} \in \{0, 1\}^*$ that aids the opening of the commitment c .
- ComVerify**($\text{srs}, f, \text{hint}, c$): checks the validity of the opening hint for the commitment c of $f \in \mathbb{F}_p^{\leq d}[x]$. If it is valid, it outputs 1; otherwise, it outputs 0.
- Open**(srs, z, s, f, d): The opening algorithm takes as input srs , an evaluation point z , a value s and the polynomial f of degree d . It outputs an opening proof π .
- OpenVerify**($\text{srs}, c, d, z, s, \pi$): The verification algorithm takes in srs , a commitment c , the degree d of the claimed polynomial, an evaluation point z , a value s and an opening proof π . It outputs 1 if π is a valid opening for (c, z, s) and 0 otherwise.

If the possibly probabilistic $\text{GenSRS}(\cdot)$ algorithm is a public coin algorithm, then we call the **PC** scheme a transparent **PC** scheme. Some formalizations of **PCs** [18, 23] define the **Open** and **OpenVerify** algorithms as interactive protocols.

A secure polynomial commitment **PC** should satisfy correctness, (polynomial) binding, evaluation binding, evaluation hiding, zero knowledge, and knowledge soundness as defined below. Additionally, a **PC** scheme might be succinct.

Definition 1 (Correct PC scheme). A **PC** scheme is correct if $\forall f \in \mathbb{F}_p^{\leq d}[x]$ and $\forall z \in \mathbb{F}_p$ the following holds:

$$\Pr \left[b_1 = b_2 = 1 \mid \begin{array}{l} \text{srs} \leftarrow \text{GenSRS}(1^\lambda, d), \\ (c, \text{hint}) \leftarrow \text{Com}(\text{srs}, f, d), \\ b_1 \leftarrow \text{ComVerify}(\text{srs}, f, \text{hint}, c), \\ s \leftarrow f(z), \\ \pi \leftarrow \text{Open}(\text{srs}, z, s, f, d), \\ b_2 \leftarrow \text{OpenVerify}(\text{srs}, c, d, z, s, \pi) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

Definition 2 ((Polynomial) binding PC scheme). A **PC** scheme is (polynomial) binding if for all PPT adversaries $\mathcal{A}$:

$$\Pr \left[b_0 = b_1 = 1 \wedge f_0 \neq f_1 \mid \begin{array}{l} \text{srs} \leftarrow \text{GenSRS}(1^\lambda, d), \\ (f_0, \text{hint}_0, f_1, \text{hint}_1, c) \leftarrow \mathcal{A}(\text{srs}), \\ b_0 \leftarrow \text{ComVerify}(\text{srs}, f_0, \text{hint}_0, c), \\ b_1 \leftarrow \text{ComVerify}(\text{srs}, f_1, \text{hint}_1, c) \end{array} \right] \leq \text{negl}(\lambda).$$

Definition 3 (Evaluation binding PC scheme). A PPT adversary $\mathcal{A}$ which outputs a commitment c and evaluation points z has at most negligible chance to open the commitment to two different evaluations s, s' . That is, let $c \in \mathbb{G}$ be the commitment, $z \in \mathbb{F}_p$ be the argument the polynomials are evaluated at, $s, s' \in \mathbb{F}_p$ the evaluations, and o, o' be the commitment openings. Then $\forall \text{PPT } \mathcal{A}$

$$\Pr \left[\begin{array}{l} \text{OpenVerify}(\text{srs}, c, z, s, o) = 1, \\ \text{OpenVerify}(\text{srs}, c, z, s', o') = 1, \\ s \neq s' \end{array} \mid (c, z, s, s', o, o') \leftarrow \mathcal{A}(\text{srs}, d) \right] \leq \text{negl}(\lambda).$$

Definition 4 (Knowledge sound PC scheme). A **PC** scheme has knowledge soundness if $\forall \text{srs}$ output by $\text{GenSRS}(1^\lambda, d)$, the (non-)interactive public-coin protocol Open is a proof of knowledge for the NP relation $\mathcal{R}_{\text{Open}}(\text{srs}, d)$ defined as follows:

$$\mathcal{R}_{\text{Open}}(\text{srs}, d) := \{((c, z, s), (f, \text{hint})) : f \in \mathbb{F}_p^{\leq d}[x] \wedge f(z) = s \wedge \wedge \text{ComVerify}(\text{srs}, f, \text{hint}, c) = 1\}. \quad (2)$$

Since most polynomial commitments are deterministic, they do not use the classical hiding definition of commitment schemes, i.e., an indistinguishability-based game for two committed values. Hence, we follow the approach of Kate, Zaverucha, and Goldberg and apply a relaxed version of hiding tailored to **PC** schemes, called evaluation hiding.

Definition 5 (Evaluation hiding PC scheme). A **PC** scheme is evaluation hiding if given $\text{srs}, \boxed{f}$ and correct opening proofs $\{(z_j, f(z_j), \pi_{\text{Open}, z_j}) : j \in [1, \lfloor \frac{\deg(f)}{2} \rfloor]\}$ for $f(x) \in \mathbb{F}_p^d[x]$ such that $\text{OpenVerify}(\text{srs}, \boxed{f}, d, z_j, f(z_j), \pi_{\text{Open}, z_j}) = 1$ for each j

- no adversary $\mathcal{A}$ can determine $f(z')$ with non-negligible probability for any unqueried index z' (computational hiding) or
- no computationally unbounded adversary $\mathcal{A}$ has any information about $f(z')$ for any unqueried index z' (unconditional hiding).

Note we define evaluation hiding for $\lfloor \frac{\deg(f)}{2} \rfloor$ opened points instead of $\deg(f)$ points. This is because each opening proof must also open the polynomial $f(\cdot)$ a random point r for knowledge soundness, cf. Section 5.3. This somewhat relaxed notion of evaluation hiding is common in the polynomial commitment literature. For example, the FRI **PC** scheme achieves a similar, relaxed evaluation hiding notion [10].

Definition 6 (Succinct PC scheme). We call a **PC** scheme succinct if both the opening proof π is constant-size and verifying the π takes constant time as well, i.e., independent from the degree of the committed polynomial.

We remark that there was no known **PC** scheme in the transparent setting that would be succinct before this work.

3.3 Generic group model adversaries

We prove the security of our scheme in the generic group model introduced by Shoup [52]. The generic group model (GGM) is an abstraction of an adversary that does not use the representation of a cryptographic group. The generic group model was first applied to groups of unknown order in 2002 by Damgård and Koprowski in [27]. Since then, it has been extensively used [1, 16, 36, 51] to show various reductions in groups of unknown order, e.g., the equivalence of the RSA and factoring assumptions [1]. Formally, we sample uniformly at random the order of the generic group (of unknown order) from the interval $[A, B]$, where $A, B \in \mathbb{N}$. Each group element is represented via an injective function $\sigma : \mathbb{Z}_{|\mathbb{G}|} \rightarrow \{0, 1\}^l$ for $2^l \gg |\mathbb{G}|$. A generic group adversary $\mathcal{A}$ is a PPT machine with access to $\mathbb{G} = \{\sigma(0), \sigma(1), \dots, \sigma(|\mathbb{G}| - 1)\}$ via the following two oracles. A list $\mathcal{L}$ is initially empty and contains the representations of group elements that $\mathcal{A}$ had queried from its oracles.

- $\mathcal{O}_1$ samples $r \in \mathbb{Z}_{|\mathbb{G}|}$ and sends $\sigma(r)$ to $\mathcal{A}$. Moreover, $\mathcal{L} := \mathcal{L} \cup \{\sigma(r)\}$.
- $\mathcal{O}_2$ allows the adversary to compute the group operation, i.e., whenever $|\mathcal{L}| = q$, $\mathcal{A}$ sends $i, j \leq q$ and a sign bit to $\mathcal{O}_2$. The oracle $\mathcal{O}_2$ returns $\sigma(x_i \pm x_j)$ to $\mathcal{A}$. Let $\mathcal{L} := \mathcal{L} \cup \{\sigma(x_i \pm x_j)\}$.

Theorem 1 (Element representation [52]). *Let $\mathbb{G}$ be a generic group, and $\mathcal{A}$ a generic algorithm making q_1 queries to $\mathcal{O}_1$ and q_2 queries to $\mathcal{O}_2$. Let $g_1, \dots, g_m$ be the outputs of $\mathcal{O}_1$. There is an efficient algorithm Ext that, given as input the transcript of $\mathcal{A}$'s interaction with the generic group oracles, produces for every element $u \in \mathbb{G}$ that $\mathcal{A}$ outputs, a tuple $(f_1, \dots, f_m) \in \mathbb{Z}^m$ such that $u = \prod_{i=1}^m g_i^{f_i}$ and $f_i \leq 2^{q+2}$.*

3.4 Assumptions in groups of unknown order

We build upon the following cryptographic assumptions in groups of unknown order.

Definition 7 (Strong RSA Assumption [7]). *Informally, the Strong RSA assumption states that no efficient adversary can compute roots of a random group element. Specifically, it holds for a group generator algorithm $\text{GGen}(\cdot)$ if for any probabilistic polynomial time adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that:*

$$\Pr \left[u^l = w, l > 1 : \begin{array}{c} \mathbb{G} \xleftarrow{\$} \text{GGen}(\lambda) \\ w \xleftarrow{\$} \mathbb{G} \\ (u, l) \leftarrow \mathcal{A}(\mathbb{G}, w) \end{array} \right] \leq \text{negl}(\lambda). \quad (3)$$

Definition 8 (Order assumption). *This assumption mandates that, given a random $g \in_R \mathbb{G}$, it is hard to find any multiple of its order: i.e., an integer l such that $g^l = 1_{\mathbb{G}}$. This is known as the order problem [45].*

$$\Pr \left[g^l = 1_{\mathbb{G}} : \begin{array}{c} \mathbb{G} \xleftarrow{\$} \text{GGen}(\lambda) \\ g \xleftarrow{\$} \mathbb{G} \\ l \leftarrow \mathcal{A}(\mathbb{G}, g) \end{array} \right] \leq \text{negl}(\lambda). \quad (4)$$

Definition 9 (Adaptive Root Assumption [58]). *For GGen if there is no efficient adversary $(\mathcal{A}_0, \mathcal{A}_1)$ that succeeds in the following task. First, $\mathcal{A}_0$ outputs an element $w \in \mathbb{G}/\{-1, 1\}$ and some state st . Then, a random prime in $\text{Primes}(\lambda)$ is chosen and $\mathcal{A}_1(w, l, st)$ outputs $w^{1/l} \in \mathbb{G}/\{-1, 1\}$. For all efficient $(\mathcal{A}_0, \mathcal{A}_1)$:*

$$\Pr \left[u^l = w \neq 1 : \begin{array}{c} \mathbb{G} \xleftarrow{\$} \text{GGen}(\lambda) \\ (w, st) \leftarrow \mathcal{A}_0(\mathbb{G}) \\ l \xleftarrow{\$} \Pi_{\lambda} = \text{Primes}(\lambda) \\ u \leftarrow \mathcal{A}_1(w, l, st) \end{array} \right] \leq \text{negl}(\lambda) \quad (5)$$

It was shown in [16] that both the order and adaptive root assumptions hold in the generic group model. Similar reductions were proven in the algebraic group model [6].

3.5 Non-interactive zero-knowledge proofs

We recall the relevant syntax of non-interactive zero-knowledge (NIZK) proofs following [14], and for the details and exact security requirements, we refer to [14]. NIZK arguments consist of four PPT algorithms that are defined with respect to a relation generator algorithm $\mathcal{R}\text{-Gen}(1^\lambda)$ that, upon receiving some security parameter λ , outputs a polynomial time decidable relation $\mathcal{R} : \{0, 1\}^* \times \{0, 1\}^*$ for which in our case $(\phi, w) \in \mathcal{R}$, where ϕ is typically an algebraic statement in a ring $\mathbb{F}_N$ or in a finite field $\mathbb{F}_p$ and w is a valid witness for the instance.

- $\text{NIZK.Setup}(\mathcal{R}) \rightarrow (\text{crs}, \tau)$. For the relation $\mathcal{R}$, the setup produces a common reference string crs and a simulation trapdoor τ . This possibly randomized algorithm may use public coins (transparent setup) or private coins (trusted setup).
- $\text{NIZK.Prove}(\mathcal{R}, \text{crs}, \phi, w) \rightarrow \pi$. Upon the $(\phi, w) \in \mathcal{R}$ and the common reference string crs , the prover returns an argument π .
- $\text{NIZK.Verify}(\mathcal{R}, \text{crs}, \phi, \pi) \rightarrow \{0, 1\}$. Upon the common reference string crs , the statement ϕ , and an argument π , the verification algorithm returns 0 or 1.
- $\text{NIZK.Sim}(\mathcal{R}, \tau, \phi) \rightarrow \pi$. Using the simulation trapdoor, τ , and statement ϕ , the simulator returns an argument π .

This paper relies on non-interactive zero-knowledge proofs built for the following NP languages in groups of unknown order. All of the corresponding proofs have a constant size and constant-time verifiers. They were shown to be proofs of knowledge in the GGM [16, 55].

Chaum-Pedersen (also known as discrete logarithm equality (DLEq)) proof [24].

$$\mathcal{R}_{\text{ChaumP}} = \{((s, t, u, v \in \mathbb{G}); x \in \mathbb{Z}) : s = t^x \wedge u = v^x\}. \quad (6)$$

Thakur showed how to instantiate the original Chaum-Pedersen discrete logarithm equality proof system in a group of unknown order setting soundly [55].

Proof of Exponentiation (PoE) [48, 58].

$$\mathcal{R}_{\text{PoE}} = \{((u, w \in \mathbb{G}, x \in \mathbb{Z}); \perp) : w = u^x \in \mathbb{G}\}. \quad (7)$$

Note there is no witness in the $\mathcal{R}_{\text{PoE}}$ relation, i.e., the verifier knows the exponent x . The primary goal of the PoE proof system for the verifier is to outsource a possibly large exponentiation, i.e., for an exponent $x \in [2^{2^{30}}, 2^{2^{50}}]$ in a group $\mathbb{G}$ of unknown order.

Proof of Knowledge of Exponent (PoKE) [16].

$$\mathcal{R}_{\text{PoKE}} = \{((u, w \in \mathbb{G}); x \in \mathbb{Z}) : w = u^x \in \mathbb{G}\}. \quad (8)$$

Note, unlike in the $\mathcal{R}_{\text{PoE}}$ relation, the verifier does not know the exponent x in the $\mathcal{R}_{\text{PoKE}}$ relation. We remark that a zero-knowledge variant of the PoKE proof system, ZKPoKE, exists due to Boneh, Bünz, and Fisch [16].

Proof of Knowledge of Exponent Modulo an odd integer (PoKEMon) [16].

$$\mathcal{R}_{\text{PoKEMon}} = \{((w, g \in \mathbb{G}, \hat{x} \in [n]); x \in \mathbb{Z}) : w = g^x \in \mathbb{G}, x \bmod n = \hat{x}\}. \quad (9)$$

Proof of Knowledge of Squared Exponent (PoKSE) [4].

$$\mathcal{R}_{\text{PoKSE}} = \{((w, g \in \mathbb{G}); x \in \mathbb{Z}) : w = g^{x^2} \in \mathbb{G}\}. \quad (10)$$

Proof of knowledge of positive exponent (PoKPE) [4].

$$\mathcal{R}_{\text{PoKPE}} = \{((w, g \in \mathbb{G}); x \in \mathbb{Z}) : (w = g^x) \wedge (0 < x)\}. \quad (11)$$

We will denote the corresponding proofs as π_{PoE} , π_{PoKE} , π_{PoKEMon} , π_{PoKPE} , π_{PoKSE} . We enclose the protocols for the aforementioned languages in Appendix A for completeness. For their proofs of security, the reader is referred to [4, 16, 48, 58]. Some of these protocols were introduced as interactive proof systems. However, all of them were shown to be secure as non-interactive proof systems in the random oracle model (ROM) using the Fiat-Shamir transformation [32]. This work uses the non-interactive version of all the aforementioned (zero-knowledge) proof systems. Thus, we assume the ROM throughout.

4 Behemoth: A Transparent, Succinct PC Scheme

This section defines the univariate Behemoth, our polynomial commitment scheme. We formally describe our **PC** scheme in Figure 1. Our Open protocol relies on two subprotocols. First, we ensure that the opening protocol is evaluation binding in the ProveEvaluation protocol; see Figure 2. Afterwards, we build a protocol that guarantees that the Behemoth-committed polynomial is of bounded degree d , see Figure 3.

4.1 Lifting polynomials over a finite field to over the rationals

We want to commit to polynomials $f \in \mathbb{F}_p^{\leq d}[x]$. However, we also want to work in a group of unknown order (GUO). Hence, we need to work over the integers in the exponent. A committer can represent every f in multiple ways. We call the canonical form of a $f \in \mathbb{F}_p^{\leq d}[x]$ polynomial, when all of its coefficients $\forall i \in [0, d] : f_i \in [0, p)$. Jumping ahead, an honest prover will always use the canonical representation of a committed polynomial. Still, we cannot force this behavior, i.e., a committer can represent internally their committed polynomial in any equivalent form. All of our protocols will work with the canonical representation of a polynomial. Let us consider the following example.

Example. Suppose we want to commit to univariate polynomials in $\mathbb{F}_7^{\leq 2}[x]$ in a group of unknown order $\mathbb{G}$. As we already alluded to, the Behemoth commitment to f will be $g^{\widehat{f}(\alpha)} \in \mathbb{G}$ for $g \in_R \mathbb{G}$ and for a carefully chosen α . For the sake of concreteness, let $\alpha := 2^5 + 1 = 33$. Let us consider $f(x) := 4(x^2 + x) \equiv \frac{x^2+x}{2} \equiv \frac{x^2+15x}{142} \pmod{7}$. All of these polynomials are equivalent $\pmod{7}$. Yet, there are crucial differences we must point out. The first polynomial $4(x^2 + x)$ is the canonical representation of the polynomial. The second representation of the polynomial $\frac{x^2+x}{2} \in \mathbb{Q}[x]$ is integer-valued everywhere and can be used to commit to the polynomial since it is an integer-valued polynomial. Note that the Behemoth commitments of $4(x^2 + x)$ and $\frac{x^2+x}{2}$ are different since $4(\alpha^2 + \alpha) \neq \frac{\alpha^2+\alpha}{2}$ even though the polynomials are equivalent $\pmod{7}$.

Nonetheless, we remark that the representation $\frac{x^2+x}{2}$ cannot be opened everywhere, as it will be apparent in Section 4.3 once we introduce our opening proofs. On the other hand, the third representation of the polynomial $\frac{x^2+15x}{142}$ is not integer-valued everywhere. In particular, $\frac{\alpha^2+15\alpha}{142} \notin \mathbb{Z}$. Specifically, suppose the committer “thinks of” the polynomial $4(x^2 + x)$ as $\frac{x^2+15x}{142}$. In that case, it cannot even commit to it unless it could compute arbitrary roots (142th roots in this example) in a group of unknown order $\mathbb{G}$, which is deemed to be computationally infeasible as long as the strong RSA assumption holds in $\mathbb{G}$, cf. Section 3.4. Motivated by this discussion, we define a homomorphism from rational polynomials to polynomials over finite fields, which maps polynomials to their canonical representations.

$$\begin{aligned} \text{Project}(\cdot) : \mathbb{Q}^{\leq d}[x] \cap \{f \mid \forall i : v_p(f_i) \geq 0\} &\rightarrow \mathbb{F}_p^{\leq d}[x]; \\ \text{Project}(f^*(x)) &:= \sum_{i=0}^d f_i x^i, \text{ such that } f_i \equiv f_i^* \pmod{p} \wedge f_i \in [0, p). \end{aligned}$$

We remark that the requirement $\{f \mid \forall i : v_p(f_i) \geq 0\}$ is necessary for the polynomials in the domain of $\text{Project}(\cdot)$. If $v_p(f_i) < 0$ was for some i , then f_i could not be mapped to $\mathbb{F}_p$. Note that $\text{Project}(\cdot)$ is a many-to-one projection, cf. the example above. Therefore, when one wants to define the “inverse” of this homomorphism, we select a canonical pre-image. This “inverse” mapping $\text{Lift}(\cdot)$ is defined as follows.

$$\text{Lift}(\cdot) : \mathbb{F}_p^{\leq d}[x] \rightarrow \mathbb{Z}^{\leq d}[x]; \text{Lift}(f) := \sum_{i=0}^d f_i x^i, \text{ such that } \forall i : f_i \in [0, p). \quad (12)$$

Provers may proceed with different representations of a polynomial in different protocols of the Behemoth **PC** scheme. We only guarantee correctness for the canonical representation of committed polynomials. If provers represent internally polynomials in a non-canonical form, then our protocols (both the commitment Com and the Open protocols) may or may not work. However, (knowledge) soundness of the protocol is guaranteed to hold for *every representative of a committed* $f \in \mathbb{F}_p^{\leq d}[x]$ polynomial for which the prover has a non-negligible probability of successfully opening it.

4.2 Behemoth: a high-level description

The Behemoth univariate polynomial commitment scheme consists of the following five PPT algorithms, cf. Figure 1. This high-level description contains two proof systems, i.e., ProveEvaluation and PoKDegUp, introduced formally in Section 4.3. These proof systems allow one to prove membership succinctly in the following NP relations.

$$\mathcal{R}_{\text{ProveEvaluation}}[f, z, s] = \{\boxed{f} \in \mathbb{G}, f(x) \in \mathbb{Q}[x], z, s \in \mathbb{F}_p : g^{\widehat{f(\alpha)}} = \boxed{f} \wedge f(z) = s\}.$$

$$\mathcal{R}_{\text{PoKDegUp}}[f, d] = \{\boxed{f} \in \mathbb{G}, f(x) \in \mathbb{Q}[x], d \in \mathbb{Z} : g^{\widehat{f(\alpha)}} = \boxed{f} \wedge \deg(f) \leq d\}.$$

The Behemoth polynomial commitment scheme

GenSRS($1^\lambda, d$): $\mathbb{G} \xleftarrow{\$} GGen(\lambda)$, $g_0 \in_R \mathbb{G}$. Let $\alpha := 2^{(d+1)(\lceil \log p \rceil + 1)} + 2^{d(\lceil \log p \rceil + 1)}$, and $g := g_0^{\alpha^{d+1}}$. Then $\text{srs}[i] := g^{\alpha^{i-(d+1)}} = g_0^{\alpha^i}$, where $i \in [0, 2(d+1)]$.

Com(srs, f, d): $\boxed{f} := g^{\widehat{f(\alpha)}} = \prod_{i=0}^d (g^{\alpha^i})^{f_i} = \prod_{i=0}^d \text{srs}[(d+1) + i]^{f_i}$, where

$$f = \sum_{i=0}^d f_i x^i \in \mathbb{F}_p^{\leq d}[x].$$

Output: $\boxed{f}$.

ComVerify($\text{srs}, f, \boxed{f}$): $\boxed{f} \stackrel{?}{=} \text{Com}(\text{srs}, f, d) \wedge f \in \mathbb{F}_p^{\leq d}[x]$.

Open(srs, f, d, z, s): The prover $\mathcal{P}$ convinces the verifier $\mathcal{V}$ that for $\boxed{f}$ it holds that $f(z) = s \wedge \deg(f) \leq d$. $\mathcal{P}$ and $\mathcal{V}$ execute the following protocols.

1. $\mathcal{P}$ runs the **ProveEvaluation**(f, z, s) protocol and sends the $\pi_{\text{ProveEvaluation}}^{f,z,s}$ to $\mathcal{V}$, see Figure 2. //This ensures that $f(z) = s$.
2. $\mathcal{P}$ samples $z' \in_R \mathbb{F}_p$ using the Fiat-Shamir transformation.
3. $\mathcal{P}$ runs the **ProveEvaluation**(f, z', s') protocol and sends the $\pi_{\text{ProveEvaluation}}^{f,z',s'}$ to $\mathcal{V}$, see Figure 2. //This step is needed for knowledge soundness and ensures that $f(z') = s'$.
4. $\mathcal{P}$ runs the **PoKDdegUp**($f(x), d$) protocol with $\mathcal{V}$, see Figure 3. //This ensures that $\deg(f) \leq d$.
5. $\mathcal{P}$ runs the **PoKDdegUp**($x^d f(1/x), d$) protocol with $\mathcal{V}$, see Figure 3. //This ensures that $f(x)$ does not contain monomials of x^{-i} for any $i \in \mathbb{Z}$.

Output $\pi_{\text{Open}} := (\pi_{\text{ProveEvaluation}}^{f,z,s}, \pi_{\text{ProveEvaluation}}^{f,z',s'}, \pi_{\text{PoKDdegUp}}^{f,d}, \pi_{\text{PoKDdegUp}}^{x^d f(1/x),d})$.

OpenVerify($\text{srs}, \boxed{f}, d, z, s, \pi_{\text{Open}}$): Parse π_{Open} as $\pi_{\text{Open}} = (\pi_{\text{ProveEvaluation}}^{f,z,s}, \pi_{\text{ProveEvaluation}}^{f,z',s'}, \pi_{\text{PoKDdegUp}}^{f,d}, \pi_{\text{PoKDdegUp}}^{x^d f(1/x),d})$.

Output: $\text{NIZK.Verify}(\mathcal{R}_{\text{ProveEvaluation}}, \text{CRS}_{\text{ProveEvaluation}}, \phi_{\text{ProveEvaluation}}, \pi_{\text{ProveEvaluation}}^{f,z,s}) \wedge$
 $\wedge \text{NIZK.Verify}(\mathcal{R}_{\text{ProveEvaluation}}, \text{CRS}_{\text{ProveEvaluation}}, \phi_{\text{ProveEvaluation}}, \pi_{\text{ProveEvaluation}}^{f,z',s'}) \wedge$
 $\wedge \text{NIZK.Verify}(\mathcal{R}_{\text{PoKDdegUp}}, \text{CRS}_{\text{PoKDdegUp}}, \phi_{\text{PoKDdegUp}}, \pi_{\text{PoKDdegUp}}^{f,d}) \wedge$
 $\wedge \text{NIZK.Verify}(\mathcal{R}_{\text{PoKDdegUp}}, \text{CRS}_{\text{PoKDdegUp}}, \phi_{\text{PoKDdegUp}}, \pi_{\text{PoKDdegUp}}^{x^d f(1/x),d})$.

Fig. 1: The formal description of the five efficient algorithms (GenSRS, Com, ComVerify, Open, OpenVerify) of the Behemoth polynomial commitment scheme for univariate polynomials.

Remarks. First, note that in the GenSRS algorithm α is public. Hence, if the setup $GGen(\lambda)$ of the underlying group of unknown order $\mathbb{G}$ does not require a trusted setup, then our **PC** scheme can be instantiated with a transparent setup. For instance, this can be achieved with class groups of imaginary quadratic fields or hyperelliptic Jacobians [29]. Furthermore, it is important that α is large (i.e., $\alpha > p^d$), (see the toy attack example with small α in Section 2), and has a low Hamming weight for efficiency reasons. Note that α cannot be a power of two since an efficient algorithm exists in class groups to compute square roots due to Gauss. If α was a power of two, our Open protocol would

not be sound. For more discussion on the choice of α , see Section 4.2. The public exponent α is large, specifically, α has $\mathcal{O}(d \log(p))$ bits, making the GenSRS algorithm a computationally heavy computation, i.e., $\mathcal{O}(d^2 \log p)$. In other words, for certain parameter settings, GenSRS essentially behaves as a verifiable delay function [15]. Practically speaking, this means that when $d \geq 2^{20}$ for larger finite fields ($p \approx 2^{256}$), the GenSRS($1^\lambda, d$) algorithm of the Behemoth **PC** scheme becomes computationally heavy, i.e., finishing the transparent setup takes several months on specialized hardware.

The size of the srs is linear in the degree d of the committed polynomial. Note that the srs needs to contain negative degrees of α in the exponent of $g^{\alpha^{-i}}$ for $i \in [1, d]$. This seems unattainable since we cannot compute α -roots in a group of unknown order $\mathbb{G}$ (we can only compute roots of powers of two in class groups) as it would contradict the strong RSA assumption, see Appendix 3.4. In practice, one would compute the powers of α in the forward direction, i.e., $g, g^\alpha, \dots, g^{\alpha^{2d}}$ and designate g^{α^d} as the “new” g . Lastly, one can attach PoE proofs [48, 58] to convince resource-constrained devices that the transparent setup was computed correctly.

Once the srs is computed, committing to $f \in \mathbb{F}_p^{\leq d}[x]$ can be in $\mathcal{O}(d)$ time as it requires d “small” exponentiations, i.e., each exponent (coefficient of f) with bit-length $\approx \log(p)$. Additionally, this computation can be parallelized.

We remark that the evaluation proof has constant size; it is independent of the degree d of the committed polynomial. This holds, as we shall see because all the applied underlying zero-knowledge proofs have constant size. Moreover, the verifier also runs in constant time. To the best of our knowledge, this is the *first transparent polynomial commitment scheme with constant evaluation proofs and constant verifier*. The verifier’s efficiency comes at the cost of a computationally heavy prover, i.e., a cubic prover. The bulk of the prover’s work comes from the difficulty of finding the three (or four) squares decomposition of $\hat{s} = \widehat{f(z)} \approx \mathcal{B}$ over the integers. This results in a cubic computation using the state-of-the-art square decomposition algorithm of Pollack and Treviño [49]. To concretely reduce the prover’s running time (unfortunately, the asymptotic complexity still remains cubic), in our construction, we decompose $\hat{s} = \epsilon + q$ to the sum of a small positive integer ϵ and a prime q that can be further decomposed to two squares much faster than decomposing $\hat{s}$ to the sum of three squares. Still, the cubic computational complexity constrains the prover to a specific range of parameters to preserve the practicality of our scheme. We further expand on our scheme’s theoretical and practical performance in Section 7.

Strong correctness. The original KZG commitment scheme satisfies the property of strong correctness, i.e., it is computationally infeasible to commit to polynomials of degrees larger than the maximum allowed degree d , i.e., the length of the srs, as long as the d -polyDH assumption holds. This property is beneficial and even wanted in certain applications, e.g., verifiable secret sharing. We note, however, that strong correctness is not satisfied by our polynomial commitment scheme as the srs is extensible by anyone. The possibility to extend the srs is valuable in certain applications, e.g., zkSNARKs. The extensible nature of the srs does not limit the complexity of the circuit one wants to prove statements about. It is conceivable that in the imminent future, when the community wants to support computational integrity proofs for statements with ever-

increasing complexity (e.g., training or inference in zero-knowledge machine learning), then currently available srs strings with length $\approx 2^{28}$ will likely fall short in supporting such complex computations. Recall that the srs cannot be extended indefinitely without increasing α since that would forfeit evaluation binding, i.e., it must hold for d and p that $\mathcal{B} = \sum_{i=1}^{d+1} (p-1)^i \leq \alpha$, see Section 5.2. However, computing a larger, updated value of $\alpha' \geq \alpha$ and updating the srs $= \{g^{\alpha^{i-(d+1)}}\}_{i=0}^{2(d+1)}$ to $\text{srs}' := \{g^{\alpha'^{i-(d+1)}}\}_{i=0}^{2(d+1)}$ accordingly can be computed with much less effort than initializing a new **PC** instance. On the other hand, a malicious prover can use the extended srs to its favor, but we prove that this does not yield an attack on the security of our polynomial commitment scheme.

On the choice of α in class groups. It is well-known that the ability to compute square roots in an RSA group, i.e., $\text{mod } N(= p \cdot q)$ with unknown factorization, is equivalent to factoring. However, in class groups, given $h_1 := g^x \in \text{Cl}(\Delta)$, one can efficiently compute the “square root of h_1 ” given the factorization of Δ (in cryptographic applications of class groups $\Delta < 0$ and prime). Specifically, given a group element h_1 , the algorithm outputs $h_2 = g^y$ such that $h_2^2 = g^{2y} = g^x$, or output $\perp$ if square roots do not exist [5]. This algorithm is due to Lagarias [41]. Assume for now that $\alpha = 2^k$.

We show next that such an α would render Behemoth insecure. If $\alpha = 2^k$, then for $z = 0, \forall s \in \mathbb{F}_p, \exists \pi_{\text{Open},s} : \text{OpenVerify}(\text{srs}, \boxed{f}, d, 0, s, \pi_{\text{Open},s}) = 1$ and such $\pi_{\text{Open},s}$ can be found efficiently. In this case, the $\text{OpenVerify}(\cdot)$ verification equation would be:

$$g^{\widehat{q(\alpha)(\alpha-z)}} = \left(g^{\widehat{q(\alpha)}}\right)^{2^k} \stackrel{?}{=} g^{\widehat{f(\alpha)}-\hat{s}}. \quad (13)$$

This check is vacuous if $\alpha = 2^k$ since the adversary can efficiently compute the 2^k -th root of any group element of $g^{\widehat{f(\alpha)}-\hat{s}}$ on the right-hand side of Equation (13). Thus, if $\alpha = 2^k$ for any $k \in \mathbb{Z}$, the **PC** scheme’s evaluation binding property would not hold.

Therefore, α cannot be a power of two, i.e., $\alpha \neq 2^k$. Similarly, α cannot be any value for which $\exists z \in \mathbb{F}_p$ such that $\alpha - \hat{z}$ is a power of two. For efficiency reasons, that is, to ensure a low Hamming weight for α , we set $\alpha = 2^k + 2^{k-1}$ for a $k \in \mathbb{Z}$ s.t. $\alpha \geq \mathcal{B}$.

4.3 Subprotocols of the Behemoth Open protocol

Next, we detail the subprotocols of our Open protocol. First, we introduce a protocol called **ProveEvaluation** that allows the prover to convince the verifier that a Behemoth-committed polynomial f is evaluated to s at z . Second, we describe a protocol that allows the prover to show that a Behemoth-committed polynomial has degree maximum d .

The ProveEvaluation protocol In this protocol, we want to prove membership in the relation $\mathcal{R}_{\text{eval}} = \{(\boxed{f}, z, s; f) \mid f(z) = s \wedge \text{Com}(\text{srs}, f, d) = \boxed{f}\}$. The goal of the **ProveEvaluation** protocol is to mimic the KZG opening strategy soundly in the group of unknown order setting. Specifically, in the KZG opening proof to show that $f(z) = s$,

the prover demonstrates that $x - z$ divides $f(x) - s$ by sending a commitment to $q(x)$ such that the following verification equation is satisfied in the exponent

$$q(x)(x - z) = f(x) - s. \quad (14)$$

The challenge in our setting is that we need to work over the integers since the order of the group is hidden. This renders our statement to be $\widehat{f(z)} = \widehat{s}$. Therefore we check the KZG opening verification Equation 14 in the exponent over the integers, i.e., we check polynomial equality in α :

$$\widehat{q(\alpha)}(\alpha - z) = \widehat{f(\alpha)} - \widehat{s}. \quad (15)$$

This strategy entails several technical challenges that we solve with techniques mainly introduced in [4, 16, 58]. In particular, the following technical challenges arise when one translates the KZG opening strategy to the group of unknown order setting:

1. The prover can only compute $\widehat{q(\alpha)}$ and not $q(\alpha) \bmod p$ in the exponent. The prover sends $Q := g^{\widehat{q(\alpha)}}$ as part of the opening proof. The prover uses the algorithm outlined in Figure 4 to compute $g^{\widehat{q(\alpha)}}$.
2. Due to efficiency reasons (recall $\alpha - z$ has $\approx d \log p$ bits), the verifier cannot compute $Q^{\alpha - z}$ from the left-hand side of the verification Equation 15 on its own, unlike in the bilinear pairing setting. This would entail a $\mathcal{O}(\log d + \log \log p)$ computation that would prevent us from achieving a constant-time verifier. Therefore, the verifier outsources this large exponentiation to the prover; that is, the prover needs to convince the verifier about the correctness of the exponentiation $Q^{\alpha - z}$ with a constant-size proof and in constant time [58].
3. On the right hand side of the verification Equation 15, the prover computes $\widehat{s}$ in the exponent, i.e., $g^{\widehat{s}}$. The prover must convince the verifier with a constant proof in constant time that $\widehat{s}$ in $g^{\widehat{s}}$ has the same remainder mod p as s , i.e., $\widehat{s} \equiv s \bmod p$. This is achieved by the PoKEMon proof introduced in [16], see Figure 7.
4. Finally, the prover shows that the exponent $\widehat{s}$ in $g^{\widehat{s}}$ lies in the appropriate range, i.e., $0 \leq \widehat{s} \leq \mathcal{B}$, recall that $\mathcal{B}$ is defined as a uniform upper bound on the integer evaluations at any polynomial, i.e., $\mathcal{B} := \max_{x \in \mathbb{F}_p, f(x) \in \mathbb{F}_p^{\leq d}[x]} \widehat{f(x)}$, cf. Section 3.1.

Lemma 1. *The ProveEvaluation protocol (cf. Figure 2) satisfies evaluation binding, i.e., it is not possible to show simultaneously that $f(z) = s \wedge f(z) = s'$ such that $s \neq s'$ for a Behemoth-committed polynomial f .*

Proof. The proof is provided in Section 5.2. ■

Note that at this point, we did not ensure that the committed polynomial f is in the desired polynomial ring $\mathbb{F}_p^{\leq d}[x]$. In particular, the committer might use polynomials of 1) higher degree than d , and 2) due to availability of negative powers of α in the exponent, i.e., $g^{\alpha^{-i}}$, in the srs, the committer might include monomials of x^{-i} in the committed polynomial. Next, we develop tools to prevent an adversarial prover from successfully opening such polynomials, i.e., with non-negligible probability.

The ProveEvaluation(f, z, s) protocol

Statement: $f(z) = s \wedge \text{Com}(\text{srs}, f, d) = \boxed{f} (= g^{\widehat{f(\alpha)}})$.

Input: $\langle \mathcal{P}(\text{srs}, f, z, s), \mathcal{V}(g^\alpha, \boxed{f}, z, s) \rangle$.

1. $\mathcal{P}$ sends $g^{\widehat{q(\alpha)}}$, where $\widehat{q(\alpha)} := \frac{\widehat{f(\alpha)} - \hat{s}}{\alpha - z}$, and $\hat{s} := \widehat{f(z)}$ over the integers. $// g^{\widehat{q(\alpha)}}$ is calculated using an algorithm in Figure 4.
2. $\mathcal{P}$ computes $g^{\widehat{q(\alpha)(\alpha-z)}}$ and obtains $\pi_{\text{PoE}}^{\alpha-z} = \text{NIZK.Prove}(\mathcal{R}_{\text{PoE}}, \text{crs}_{\text{PoE}}, (g^{\widehat{q(\alpha)}}, g^{\widehat{q(\alpha)(\alpha-z)}}, \alpha - z), \perp)$.
 - In the verification of $\pi_{\text{PoE}}^{\alpha-z}$, the verifier would need to compute $\alpha - z \bmod p' = \alpha \bmod p' - z \bmod p'$, where $p' \in_R \text{Primes}(\lambda)$, see Wesolowski's proof of exponentiation protocol in Appendix A.2. Computing $\alpha \bmod p'$ would entail computing $\mathcal{O}(\log d)$ multiplications in $\mathbb{Z}_{p'}^*$. Hence, the verifier outsources this computation to the prover to avoid this logarithmic computation. The correctness of this outsourced computation is proved by $\pi_{\text{PoKEMon}}^{\alpha, \alpha \bmod p'} := \text{NIZK.Prove}(\mathcal{R}_{\text{PoKEMon}}, \text{crs}_{\text{PoKEMon}}, (g^\alpha, g^{\alpha \bmod p'}, \alpha)$.
3. $\mathcal{P}$ sends $g^{\hat{s}}$ to the verifier $\mathcal{V}$ and calculates $\pi_{\text{PoKEMon}}^{s, \hat{s}} := \text{NIZK.Prove}(\mathcal{R}_{\text{PoKEMon}}, \text{crs}_{\text{PoKEMon}}, (g^s, g^{\hat{s}}, \hat{s})$. $//$ This ensures that $s \equiv \hat{s} \bmod p$.
4. Let $J := g^{\mathcal{B}}/g^{\hat{s}}$. The prover creates the proof $\pi_{\text{PoKPE}}^{\mathcal{B}-\hat{s}} := \text{NIZK.Prove}(\mathcal{R}_{\text{PoKPE}}, \text{crs}_{\text{PoKPE}}, J, \mathcal{B} - \hat{s})$ and also computes $\pi_{\text{PoKPE}}^{\hat{s}} := \text{NIZK.Prove}(\mathcal{R}_{\text{PoKPE}}, \text{crs}_{\text{PoKPE}}, g^{\hat{s}}, \hat{s})$. $//$ These proofs ensure that $0 \leq \hat{s} \leq \mathcal{B}$.

The proof: $\pi_{\text{ProveEvaluation}}^z := (\pi_{\text{PoE}}^{\alpha-z}, \pi_{\text{PoKEMon}}^{\alpha, \alpha \bmod p'}, \pi_{\text{PoKEMon}}^{s, \hat{s}}, \pi_{\text{PoKPE}}^{\mathcal{B}-\hat{s}}, \pi_{\text{PoKPE}}^{\hat{s}})$.

Verification: Parse the $\pi_{\text{ProveEvaluation}}^z$ proof as the tuple $\pi_{\text{ProveEvaluation}}^z = (\pi_{\text{PoE}}^{\alpha-z}, \pi_{\text{PoKEMon}}^{\alpha, \alpha \bmod p'}, \pi_{\text{PoKEMon}}^{s, \hat{s}}, \pi_{\text{PoKPE}}^{\mathcal{B}-\hat{s}}, \pi_{\text{PoKPE}}^{\hat{s}})$.

Output: $\text{NIZK.Verify}(\mathcal{R}_{\text{PoE}}, \text{crs}_{\text{PoE}}, \phi_{\text{PoE}}, \pi_{\text{PoE}}^{\alpha-z}) \wedge$

$\wedge \text{NIZK.Verify}(\mathcal{R}_{\text{PoKEMon}}, \text{crs}_{\text{PoKEMon}}, \phi_{\text{PoKEMon}}, \pi_{\text{PoKEMon}}^{\alpha, \alpha \bmod p'}) \wedge$

$\wedge \text{NIZK.Verify}(\mathcal{R}_{\text{PoKEMon}}, \text{crs}_{\text{PoKEMon}}, \phi_{\text{PoKEMon}}, \pi_{\text{PoKEMon}}^{s, \hat{s}}) \wedge$

$\wedge \text{NIZK.Verify}(\mathcal{R}_{\text{PoKPE}}, \text{crs}_{\text{PoKPE}}, \phi_{\text{PoKPE}}, \pi_{\text{PoKPE}}^{\mathcal{B}-\hat{s}}) \wedge$

$\wedge \text{NIZK.Verify}(\mathcal{R}_{\text{PoKPE}}, \text{crs}_{\text{PoKPE}}, \phi_{\text{PoKPE}}, \pi_{\text{PoKPE}}^{\hat{s}})$.

Fig. 2: Behemoth ProveEvaluation protocol formal description. In the ProveEvaluation protocol, the prover convinces the verifier that $f(z) = s$. It is a subprotocol of the Behemoth Open protocol, cf. Figure 1.

Proving a degree bound of the Behemoth-committed polynomial A crucial part of the Open protocol is to ensure that the committed polynomial $f \in \mathbb{F}_p^{\leq d}[x]$ has a degree less than or equal to d . This is not immediate in our setting, unlike in the KZG setting. Specifically, in the KZG PCS, if the srs has length d , then an efficient prover *cannot* commit to polynomials of degree larger than d as long as the d -polyDH assumption holds. This is because the exponents in the KZG srs are hidden, thanks to the trusted setup. However, in our setting, anyone can freely extend the srs to be able to support larger degree polynomials. Therefore, we must deal with malicious provers that can commit to arbitrarily large degree polynomials; $\deg(f) \in \mathcal{O}(\text{poly}(\lambda))$. We follow the

footsteps of Thakur and adapt his zero-knowledge proof systems about KZG-committed polynomials introduced in [54] to the group of unknown order setting.

Proof of knowledge of a polynomial with a degree upper bound (PoKDdegUp). The main goal of this subsection is to build a proof system that can show that a Behemoth-committed polynomial has degree most d using the previously introduced building blocks, i.e., PoKE, PoKPE, ProveEvaluation. We adapt Thakur’s corresponding proof system for KZG-committed polynomials [54] to Behemoth-committed polynomials.

Thakur observes that $\forall d \in \mathbb{N}, \forall f \in \mathbb{Z}[x] : \deg(f) \leq d \iff x|x^{d+1} \cdot f(x^{-1})$. Note the verifier already knows a commitment to x^{d+1} from the srs, i.e., let $a := \boxed{x^{d+1}}$ be the corresponding srs commitment to the monomial of degree $d + 1$. The prover then sends a Behemoth-commitment to $b := \boxed{x^{d+1}f(x^{-1})}$ along with a PoKE(g^α, b) that shows that b is a commitment to a polynomial divisible by x . This is where it is important that α is not a power of two. Otherwise, this PoKE(g^α, b) proof is vacuous since, in class groups, it is possible to compute the roots of powers of two due to Lagarias [41]. We remark that we need the “negative” powers of α in the srs to commit to $f(1/x)$. Now, the prover shows the well-formedness of commitment b , i.e., that it is indeed a commitment to $x^{d+1}f(x^{-1})$. For a randomly generated challenge $\gamma \in_R \mathbb{F}_p^*$, the prover verifiably sends the element $g^{\widehat{f(\gamma)}}$, along with an evaluation proof that this is a commitment to the evaluation of $f(x)$ at γ , see the ProveEvaluation protocol at Figure 2. The prover also sends the element $c := \boxed{x^{d+1}\widehat{f(\gamma)}} = g^{\alpha^{d+1}\widehat{f(\gamma)}}$ along with a Chaum-Pedersen proof [24] (also known as the discrete logarithm equality (DLEq) proof) to show that the discrete logarithms between the pair $(g, g^{\widehat{f(\gamma)}})$ and the pair $(g^{\alpha^{d+1}}, c)$ are the same, namely the common discrete logarithm is $\widehat{f(\gamma)}$. Next, the prover shows that the polynomial $h(x) := x^{d+1}f(x^{-1})$ committed in b satisfies the following relation:

$$\gamma h(x) \equiv \gamma x^{d+1}f(\gamma) \pmod{(\gamma x - 1)}. \quad (16)$$

The prover does so by producing a $(\gamma\alpha - 1)$ -th root of $b^\gamma \cdot c^{-\gamma} = g^{\gamma\alpha^{d+1}\widehat{f(1/\alpha)} - \gamma\alpha^{d+1}\widehat{f(\gamma)}}$. Since γ is randomly and uniformly generated, this implies that with overwhelming probability, $h(x) = x^{d+1} \cdot f(x^{-1})$, due to the order assumption, see Section 3.4. Hence, the following proof system is an honest verifier zero-knowledge proof for the following relation:

$$\mathcal{R}_{\text{PoKDdegUp}}[f, d] = \{\boxed{f} \in \mathbb{G}, f(x) \in \mathbb{Q}[x], d \in \mathbb{Z} : g^{\widehat{f(\alpha)}} = \boxed{f}, \deg(f) \leq d\}.$$

Example. What would constitute a soundness break of the PoKDdegUp protocol? Consider the polynomial $f(x) = x^2 + 3\alpha^3$. This polynomial has the same Behemoth-commitment as $g(x) = 3x^3 + x^2$. From the verifier’s perspective, it is indistinguishable which of these polynomials f or g are “in the prover’s head”. However, it is easy to see that the prover can only run successfully the ProveEvaluation(f, z, s) protocol for any $z \in \mathbb{F}_p$ with the evaluations of $s := g(z)$ due to the applied range checks in the ProveEvaluation protocol. We call such two polynomials *evaluation equivalent*, cf.

The PoKDegUp(f, d) protocol

Statement: The prover $\mathcal{P}$ knows a polynomial $f(x) \in \mathbb{Q}[x]$ such that $\boxed{f} = g^{\widehat{f(\alpha)}} \wedge \deg(f) \leq d$.

Input: $\langle \mathcal{P}(\text{srs}, f, d, \boxed{f}), \mathcal{V}(g^{\alpha^{d+1}}, d, \boxed{f}) \rangle$.

1. $\mathcal{P}$ sends the element $b := g^{\alpha^{d+1} \cdot f(1/\alpha)}$ with a proof for $\text{PoKE}[g^\alpha, b]$.
2. $\mathcal{P}$ samples a challenge $\gamma \in_R \mathbb{F}_p^*$ using the Fiat-Shamir transformation.
3. $\mathcal{P}$ sends the elements $g^{\widehat{f(\gamma)}}$ and $c := g^{\alpha^{d+1} \widehat{f(\gamma)}}$ with a Chaum-Pedersen DLEq proof for the pairs $(g, g^{\widehat{f(\gamma)}})$ and $(g^{\alpha^{d+1}}, c)$.
4. $\mathcal{P}$ runs the protocol $\text{ProveEvaluation}(f, \gamma, f(\gamma))$, i.e., the prover convinces the verifier that $f(\gamma)$ evaluates to a certain value.
5. $\mathcal{P}$ sends proofs for $\text{PoKE}[g^{\alpha-\gamma}, g^{\widehat{f(\alpha)} - \widehat{f(\gamma)}}]$ and $\text{PoKE}[g^{\gamma\alpha-1}, b^\gamma c^{-\gamma}]$.

The proof: $\pi_{\text{PoKDegUp}}^{f,d} := (\text{PoKE}[g^\alpha, b], \text{ChaumP}[g, g^{\widehat{f(\gamma)}}, g^{\alpha^{d+1}}, c],$

$\text{ProveEvaluation}[f, \gamma, f(\gamma)], \text{PoKE}[g^{\alpha-\gamma}, g^{\widehat{f(\alpha)} - \widehat{f(\gamma)}}], \text{PoKE}[g^{\gamma\alpha-1}, b^\gamma c^{-\gamma}].$

Verification: $\mathcal{V}$ verifies the ProveEvaluation , PoKE , and the Chaum-Pedersen DLEq proofs.

Fig. 3: Proof of knowledge of a polynomial with degree upper bounded (PoKDegUp). The prover convinces the verifier that the degree of the Behemoth-committed polynomial f is upper bounded by an integer d . PoKDegUp is a subprotocol of the Behemoth Open protocol, cf. Figure 1.

Definition 10. Motivated by this discussion, we want the prover not to be able to prove that its committed polynomial f has $\deg(f) \leq 2$. It is easy to see that this holds in this simple case. If the prover wants to show that $\deg(f) \leq 2$ in the commitment $\boxed{f}$, then $f(1/x) = \frac{1}{x^2} + 3\alpha^3$. In the last step of the PoKDegUp protocol, the prover must show that $\gamma\alpha - 1 | \gamma f_d \alpha^{d+1} \widehat{f(1/\alpha)} - \gamma f_d \alpha^{d+1} \widehat{f(\gamma)}$. In this particular example, this check entails to $\gamma\alpha - 1 | \gamma \alpha^3 (\frac{1}{\alpha^2} + 3\alpha^3) - \gamma \alpha^3 (\gamma^2 + 3\gamma^3) = \gamma\alpha(1 + 3\alpha^5) - \gamma\alpha(\gamma^2 \alpha^2 + 3\gamma^3 \alpha^2)$. Since $\gcd(\gamma\alpha - 1, \gamma\alpha) = 1$, therefore $\gamma\alpha - 1 | 3\alpha^5 - 3\alpha^2 \gamma^3 - (\gamma\alpha + 1)(\gamma\alpha - 1) \iff \gamma\alpha - 1 | 3\alpha^5 - 3\alpha^2 \gamma^3 = 3\alpha^2(\alpha^3 - \gamma^3) \iff (\gamma\alpha - 1) | (\alpha^3 - \gamma^3)$. Since $\gcd(\gamma\alpha - 1, \alpha^3) = 1$, we have that $(\gamma\alpha - 1) | (\alpha^3 - \gamma^3) \iff (\gamma\alpha - 1) | \alpha^3(\alpha^3 - \gamma^3) = \alpha^6 - \gamma^3 \alpha^3$. Because of $(\gamma\alpha - 1) | \gamma^3 \alpha^3 - 1$, it suffices to show that $(\gamma\alpha - 1) | \alpha^6 - 1$, where the right-hand side does not contain γ anymore, that is chosen by the verifier uniformly at random. We conclude that the prover cannot convince the verifier that $f(x) = x^2 + 3\alpha^3$ is a quadratic polynomial. Next, we prove that the PoKDegUp protocol is secure in full generality.

Lemma 2. *The PoKDegUp protocol, see Figure 3, is knowledge sound for the relation $\mathcal{R}_{\text{PoKDegUp}} = \{(\boxed{f}, d; f) | f \in \mathbb{Q}[x] \wedge \deg(f) \leq d\}$ in the generic group and random oracle models.*

Proof. Suppose a PPT algorithm $\mathcal{A}$ outputs an accepting transcript. The extractability of the subprotocols $\text{PoKE}[g^{\alpha-\gamma}, g^{\widehat{f(\alpha)} - \widehat{f(\gamma)}}]$ and $\text{PoKE}[g^{\gamma\alpha-1}, b^\gamma c^{-\gamma}]$ imply that with overwhelming probability, $\mathcal{A}$ can output polynomials $f(x), g(x) = x^{d+1} \widehat{f(\gamma)}, h(x) =$

$x^{d+1}f(1/x)$ such that

$$g^{\widehat{f(\alpha)}} = \boxed{f}, g^{\widehat{g(\alpha)}} = g^{\alpha^d \widehat{f(\gamma)}} = b, g^{\widehat{f(\gamma)}}, g^{\widehat{h(\alpha)}} = c, \gamma h(x) \equiv \gamma g(x) \pmod{(\gamma x - 1)}. \quad (17)$$

Since we do not assume the Knowledge of Exponent Assumption (KEA), we cannot immediately claim that such polynomials can be extracted from the prover. Instead, we argue as follows. Due to the extractability of the applied NIZK proof systems, the prover must *know* an integer in the exponent for every Behemoth commitment. Therefore, whenever we say that the prover knows a polynomial for a commitment, we refer to the unique polynomial that is derived from the α -adic representation of the integer known by the prover in the exponent. The Chaum-Pedersen DLEq proof implies that

$$g^{\alpha^{d+1} \widehat{f(\gamma)}} = a^{\widehat{f(\gamma)}} = c = g^{\widehat{h(\alpha)}} \quad (18)$$

Thus, the verifier checks in the exponent that

$$x^{d+1}f(x^{-1}) \equiv g(x) \pmod{(\gamma x - 1)}, \quad (19)$$

and since γ was randomly and uniformly generated, this implies that with overwhelming probability $g(x) = x^{d+1}f(x^{-1})$ holds. This is because if it was the case that $g(x) \neq x^{d+1}f(x^{-1})$ and there was a non-negligible probability of finding a suitable γ for which $g(\gamma) \equiv \gamma^{d+1}f(\gamma^{-1}) \pmod{\text{ord}(\mathbb{G})}$, then $g(\gamma) - \gamma^{d+1}f(\gamma^{-1}) \equiv 0 \pmod{\text{ord}(\mathbb{G})}$ contradicting the order assumption, see Section 3.4. Finally, the subprotocol PoKE $[g^\alpha, b]$ implies that with overwhelming probability, $g(x)$ is divisible by x , hence it follows that $\deg(f) \leq d$.

The only thing remaining to show is the case when the prover does not use the canonic α -adic polynomial representation of a polynomial. Let $\deg_\alpha(f)$ be the degree of the canonical α -adic representation of the Behemoth-committed polynomial $\boxed{f}$. On the other hand, let $\deg^*(f)$ be the degree of the malicious prover's interpretation of $\boxed{f}$, where $\deg^*(f) \neq \deg_\alpha(f)$. Due to the correctness of this protocol, the following check for the honest prover needs to be satisfied,

$$(\gamma\alpha - 1) | \gamma h(\alpha) - \alpha^{\deg_\alpha(f)+1} \gamma \widehat{f(\gamma)}. \quad (20)$$

Now, suppose that the prover applies a different polynomial h' in the check, that is,

$$(\gamma\alpha - 1) | \gamma h'(\alpha) - \alpha^{\deg^*(f)+1} \gamma \widehat{f(\gamma)}. \quad (21)$$

If we subtract Equation 21 from Equation 20, then we find that the malicious prover must satisfy the following division,

$$(\gamma\alpha - 1) | \gamma(h(\alpha) - h'(\alpha)) - \gamma(\alpha^{\deg_\alpha(f)} - \alpha^{\deg^*(f)}) \widehat{f(\gamma)}, \quad (22)$$

where $\gcd(\gamma\alpha - 1, \gamma) = 1$, thus we can eliminate γ . Note that $\widehat{f(\gamma)}$ can be opened to a single evaluation due to the evaluation binding property of the ProveEvaluation protocol. Towards contradiction, assume that $\alpha^{\deg_\alpha(f)} \neq \alpha^{\deg^*(f)}$. We have the following congruence for $\widehat{f(\gamma)}$,

$$\widehat{f(\gamma)} \equiv \frac{h(\alpha) - h'(\alpha)}{\alpha^{\deg_\alpha(f)} - \alpha^{\deg^*(f)}} \pmod{(\gamma\alpha - 1)}. \quad (23)$$

Both the nominator $(h(\alpha) - h'(\alpha))$ and the denominator $\alpha^{\deg_\alpha(f)} - \alpha^{\deg^*(f)}$ are established before $\gamma \in_R \mathbb{F}_p$ is sent to the prover from the verifier or equivalently in the random oracle model, sampled via the Fiat-Shamir transformation [32]. Since $\widehat{f(\gamma)}$ is proved by the committer to be the evaluation of f at γ , and the evaluation binding property of the ProveEvaluation protocol has been shown, we conclude that Equation 23 can be satisfied only with negligible probability. ■

Flexible Behemoth Open proofs over multiple fields. Since the Behemoth-commitment of a polynomial f commits to it over the integers, one can reuse a commitment to later open the same commitment over different fields $\mathbb{F}_p$ and $\mathbb{F}_{p'}$. This flexibility might have several applications, as discussed next. Observe that all steps except one in the whole Open protocol are oblivious to the choice of the field $\mathbb{F}_p$ over which the committed polynomial is opened. The sole exception is the third step in the ProveEvaluation protocol, namely where the prover shows for the opening statement $f(z) = s$ that $\widehat{s} \equiv s \pmod p$ holds with a PoKEMon proof, cf. Figure 2 and Figure 7. If the p -dependent bound $\mathcal{B}$ for evaluations of polynomials (cf. Equation 1) remains smaller than α , one can safely open a Behemoth-committed polynomial over that field $\mathbb{F}_p$ as well.

5 Security Proofs

In this section, we prove the security of our polynomial commitment scheme.

Theorem 2. (*Behemoth is a secure PC scheme*) *The Behemoth PC scheme is a secure, succinct PC scheme, i.e., it satisfies correctness (cf. Section 5.1), evaluation binding (cf. Section 5.2), knowledge soundness (cf. Section 5.3), polynomial binding (cf. Section 5.4), and evaluation hiding (cf. Section 5.5) in the generic group and random oracle models.*

Concrete assumptions and the generic group model for GUOs Even though the generic group model (GGM) implies most of the applied concrete assumptions (e.g., order assumption, discrete logarithm assumption, adaptive root assumption), in the following, we spell out the concrete assumptions used in individual proofs as well. This demonstrates the minimal assumptions necessary to prove certain security properties of our PC scheme. Note we only assume the GGM for knowledge soundness. The other properties (i.e., correctness, polynomial binding, evaluation binding, and evaluation hiding) are reduced to concrete assumptions.

Generic group model for class groups We refrained from introducing a new oracle in the GGM to model the ability to compute square roots in class groups of imaginary quadratic fields; see Section 4.2. Related works that use the GGM for class groups also do not model this added capability in their security modelling for multiple reasons [6, 16]. First, an added square root oracle would make the class group GGM variant weaker. Second, in all Behemoth subprotocols, the prover is forced to compute e -th roots of class group elements for a randomly chosen prime e . Computing random e -th roots in groups

of unknown order is equivalent to the order assumption in the GGM [5,6]. Third, in our protocols, we purposefully avoided any powers of two in the exponent (see Section 4.2) exactly to counter these square root computing capabilities in class groups.

5.1 Correctness

Lemma 3. *The Behemoth PC scheme satisfies correctness.*

Proof. The OpenVerify($\cdot$) algorithm checks whether $Q^{\alpha-z} = \boxed{f}/g^{\hat{s}}$, for $Q = g^{\widehat{q(\alpha)}}$ and $\boxed{f} = g^{\widehat{f(\alpha)}}$. In particular, the verifier checks the polynomial equality $\widehat{q(\alpha)}(\alpha - z) = \widehat{f(\alpha)} - \widehat{f(z)}$ in the exponent. The additional NIZKs ensure the soundness of the Open protocol, i.e., range checks, equality checks mod p , and degree bound on the committed polynomial. These applied NIZKs are as follows: for the languages $\mathcal{R}_{\text{ChauMP}}, \mathcal{R}_{\text{PoE}}, \mathcal{R}_{\text{PoKE}}, \mathcal{R}_{\text{PoKEMon}}, \mathcal{R}_{\text{PoKPE}}, \mathcal{R}_{\text{ZKPoKPE}}, \mathcal{R}_{\text{PoKDegUp}}$. All these NIZKs were proved to satisfy correctness in prior work [4, 16, 48, 58], and in Section 4.3. We apply the *probabilistic* Pollack-Treviño algorithm [49] in the Lagrange four-square decomposition in the proof of knowledge of positive exponent protocol (PoKPE, cf. Appendix A.5). Therefore, we can only claim correctness with probability at least $1 - \text{negl}(\lambda)$ for some negligible function $\text{negl}(\cdot)$. ■

5.2 Evaluation binding

Lemma 4. *The ProveEvaluation protocol (cf. Figure 2) satisfies evaluation binding if the order assumption holds in $\mathbb{G}$, i.e., it is not possible for a PPT adversary to show that $f(z) = s \wedge f(z) = s'$ such that $s \neq s'$ for a Behemoth-committed polynomial f .*

Proof. Adversary $\mathcal{A}_1$ in its security game needs to compute the order of the GUO $\mathbb{G}$. To that end, $\mathcal{A}_1$ instantiates the Behemoth PC protocol in the GUO $\mathbb{G}$ and invokes an adversary $\mathcal{A}_0$ breaking the evaluation binding property of Behemoth to compute $|\mathbb{G}|$.

Towards contradiction, assume there is an efficient adversary $\mathcal{A}_0$ who breaks evaluation binding with non-negligible probability, i.e., $\mathcal{A}_0$ outputs an evaluation point z for which $\mathcal{A}_0$ knows $s_0, s_1, (s_0 \neq s_1)$ and $\pi_{\text{Open},0}, \pi_{\text{Open},1}$ asserting that $f(z) = s_0$ and $f(z) = s_1$ respectively, and both statements and proofs are accepted by the verifier. Formally, this entails that the following two equalities are verified successfully in the exponent:

$$\widehat{q_0(\alpha)}(\alpha - z) = \widehat{f(\alpha)} - \hat{s}_0 \wedge \widehat{q_1(\alpha)}(\alpha - z) = \widehat{f(\alpha)} - \hat{s}_1, \quad (24)$$

where $s_0 = \hat{s}_0 \bmod p, s_1 = \hat{s}_1 \bmod p$, i.e., the evaluations of f of z over the integers. Subtracting these two equalities in Equation 24 from one another we get:

$$(\widehat{q_0(\alpha)} - \widehat{q_1(\alpha)})(\alpha - z) = \hat{s}_1 - \hat{s}_0. \quad (25)$$

The right side of Equation 25 is non-zero by assumption and $\hat{s}_1 - \hat{s}_0 \in [-\mathcal{B}, \mathcal{B}]$. On the other hand, by assumption $\widehat{q_0(\alpha)} \neq \widehat{q_1(\alpha)}$ and by construction $(\alpha - z) \gg \mathcal{B}$. Therefore, the left-hand side of Equation 25 never falls into the interval $\hat{s}_1 - \hat{s}_0 \in [-\mathcal{B}, \mathcal{B}]$, i.e., they cannot be equal over the integers. If Equation 25 is satisfied, then $(\widehat{q_0(\alpha)} - \widehat{q_1(\alpha)})(\alpha - z)$

$z) - (\hat{s}_1 - \hat{s}_0) \equiv 0 \pmod{\text{ord}(\mathbb{G})}$, hence breaking the order assumption. In other words, $\mathcal{A}_1$ can break the order assumption using $\mathcal{A}_0$ as a black box. The success probability and running time of $\mathcal{A}_1$ breaking the order assumption is the same as those of $\mathcal{A}_0$. ■

5.3 Knowledge soundness

Due to the transparent nature of our PCS, it is inherent that a prover is only bound to $\widehat{f(\alpha)}$ rather than a unique polynomial $f \in \mathbb{F}_p^{\leq d}[x]$. At first, this seems to violate knowledge soundness. However, since evaluation binding holds, see Section 5.2, the map $g : z \mapsto f(z) \pmod p$ is set to stone after committing to f . Below we show that the existence of this map g implies the knowledge of a $\pmod p$ polynomial f^* in the desired polynomial ring $\mathbb{F}_p^{\leq d}[x]$.

Consider $\boxed{f} := g^\alpha$, it may seem ambiguous whether it is a commitment to $f(x) = x$ or $f(x) = \alpha$. We remark that in the ProveEvaluation protocol, the prover can only open the $f(x) = x$ polynomial. This is because the applied range proofs in the ProveEvaluation protocol, i.e., the prover needs to show that at the evaluated point z , we have that $0 \leq \widehat{f(z)} \leq \mathcal{B}(< \alpha)$. Therefore, it is clear that the prover can only evaluate the $f(x) = x$ polynomial for every $z \in \mathbb{F}_p$. Note that without range proofs on $\widehat{f(z)}$ in the ProveEvaluation protocol, the prover could convince the verifier about the validity of any $\widehat{f(z)} + h(z)(\alpha - z)$ value evaluated of the committed polynomial at z . Thanks to the applied range proof, the committer can only evaluate the commitment $\boxed{f}$ as $f(z) = z$, this being the smallest positive representative of $\widehat{f(z)} \pmod{\alpha - z}$. This observation motivates the following definition.

Definition 10 (Evaluation Proxy). *The polynomial $f_1 \in \mathbb{Q}^{\leq d}[x]$ is an evaluation proxy for $f_0 \in \mathbb{Q}^{\leq d}[x]$, i.e., $f_0 \rightarrow_{\text{Eval}} f_1$ if $\forall z : \text{OpenVerify}(\text{srs}, \boxed{f_0}, z, f_0(z), \pi_{\text{Open}}^{f_0, z}) = 1; \widehat{f_0(z)} \equiv \widehat{f_1(z)} \pmod{\alpha - z}$. Put differently, $\widehat{f_0(z)} = \widehat{f_1(z)} + h(z)(\alpha - z)$, for some $h(z) \in \mathbb{Z}$. In other words, whenever f_0 is opened successfully at z , it has the same evaluation as f_1 at z . If $f_0 \rightarrow_{\text{Eval}} f_1 \wedge f_1 \rightarrow_{\text{Eval}} f_0$ holds, then we say that f_0 and f_1 are evaluation equivalent, i.e., $f_0 \equiv_{\text{Eval}} f_1$.*

Example. $\alpha^2 + 3\alpha \rightarrow_{\text{Eval}} x^2 + 3x$. They are different polynomials $\pmod p$, but they behave the same from an evaluation point of view. Even though the former polynomial can be considered a constant polynomial, anywhere the committer can open $\alpha^2 + 3\alpha$, it will “behave” as a polynomial $x^2 + 3x$. For every $z \in \mathbb{F}_p$, the committer can create convincing ProveEvaluation proofs such that $f(z) = z^2 + 3z$. The previously proved evaluation binding property implies that this is the only way the prover can open a Behemoth commitment of a polynomial in $\mathbb{Q}[x]$. Similarly, it is easy to see that $x^2 + 3x \rightarrow_{\text{Eval}} \alpha^2 + 3\alpha$ holds.

Example 2. $\frac{\alpha^2 + \alpha \cdot x}{2} \rightarrow_{\text{Eval}} x^2$. One can only open successfully $\frac{\alpha^2 + \alpha \cdot x}{2}$ at points where $\alpha \equiv z \pmod 2$. And at any successfully openable point z , $\frac{\alpha^2 + \alpha \cdot x}{2}$ has exactly the same evaluation as x^2 . Observe that, on the other hand, $x^2 \not\rightarrow_{\text{Eval}} \frac{\alpha^2 + \alpha \cdot x}{2}$, since there exist points z where x^2 can be opened, while for $\frac{\alpha^2 + \alpha \cdot x}{2}$ could not be opened.

Polynomials that can be opened at least at a single point z with non-negligible probability will play a crucial role in our knowledge soundness proof. Recall from Figure 1, steps (2) and (3) in the Open protocol, this means that the prover can successfully run the ProveEvaluation protocol with non-negligible probability for a randomly chosen $z' \in \mathbb{F}_p$. We define openable polynomials formally as follows.

Definition 11 (Openable polynomials). *Given a valid srs, a polynomial $f \in (\mathbb{Q} \cap \mathbb{Z}_p)^{\leq d}[x]$ is said to be openable if*

$$\exists z \exists s \exists \pi_{\text{Open}}^{*,f,z} \forall \text{negl}(\lambda) : \Pr[\text{OpenVerify}(\text{srs}, \boxed{f}, d, z, s, \pi_{\text{Open}}^{*,f,z}) = 1] \geq \text{negl}(\lambda), \quad (26)$$

where $\pi_{\text{Open}}^{*,f,z}$ a maliciously generated opening proof for f, z, s .

As we saw in the proof of correctness in Section 5.1, honest provers can always convince the verifier about correct openings. However, as noted above, we cannot force provers to use the canonical lifts of polynomials in $\mathbb{F}_p^{\leq d}[x]$. For instance, consider the polynomial of $2x^3 - x + 7$. Due to its linear term $\text{Lift}(\text{Project}(2x^3 - x + 7)) \neq 2x^3 - x + 7$, but the prover can open everywhere the Behemoth commitment of $2x^3 - x + 7$. It is easy to argue that the opened values are always the same $\bmod p$ as for $\text{Lift}(\text{Project}(2x^3 - x + 7)) = 2x^3 + (p-1)x + 7$, as expected. Thus, $\bmod p$ knowledge soundness holds.

Definition 12 (p -faithful polynomial). *A polynomial $f \in \mathbb{Q}^{\leq d}[x]$ is said to be p -faithful if $\forall z \in \mathbb{F}_p$ where f can be opened with non-negligible probability, the evaluation $f(z) = s \equiv \text{Project}(f)(z) \bmod p$.*

By the design of the ProveEvaluation protocol, all polynomials in $\mathbb{F}_p^{\leq d}[x]$ are p -faithful. As the example above shows, there are polynomials in $\mathbb{Q}^{\leq d}[x] \setminus \mathbb{F}_p^{\leq d}[x]$ that are p -faithful. What poses a challenge to proving knowledge soundness is that not all openable polynomials are p -faithful polynomials.

Example. $f(x) = \alpha$. Observe that f is not p -faithful. Although it is openable everywhere and its evaluations are congruent $\bmod p$ to the $f(x) = x$ polynomial. Yet, the commitment to the f polynomial could be viewed as a commitment to a constant polynomial. Even if the prover considers the committed polynomial a constant, it can only open it as the identity function of $\mathbb{F}_p$. This does not contradict knowledge soundness since the prover's behavior is identical to a prover who considers the committed polynomial as $f(x) = x$.

As we shall show, the opening behavior of any openable polynomial can be reproduced by a p -faithful polynomial.

Lemma 5. *If $f_0 \in \mathbb{Q}^{\leq d}[x]$ is an openable polynomial, then there exists f^* such that $f_0 \rightarrow_{\text{Eval}} f^*$ and f^* is p -faithful.*

Proof. If f_0 is an openable polynomial, we rewind $d + 1$ times the execution of the Open protocol to immediately before step (2), cf. Figure 1, with fresh randomness $z_i \in_R \mathbb{F}_p$ in each round $i \in [0, d]$. Every round $\forall i : i \in [0, d]$, the prover must also run the ProveEvaluation(f_0, z_i, s_i) protocol. Thus, the extractor obtains with non-negligible

probability, two vectors $T := \{z_i\}_{i=0}^d$ and $\{s_i\}_{i=0}^d$ such that $\forall i \in [0, d] : f_0(z_i) = s_i$ holds. The soundness of the PoKDegUp protocol, cf. Section 4.3, ensures that f_0 is a maximum d degree polynomial. Therefore, next, the extractor Lagrange-interpolates a degree d polynomial $f^* \in \mathbb{Q}^{\leq d}[x]$ given the obtained valid evaluations from the rewinding process. These evaluations are unique due to evaluation binding proved in Section 5.2. Hence, the Lagrange-interpolation f^* of f has the form:

$$\begin{aligned} f^*(x) &:= \sum_{i=0}^d s_i \mathcal{L}_i(x) = \sum_{i=0}^d s_i \prod_{j=0, j \neq i}^d \frac{x - z_j}{z_i - z_j} \leq (p-1) \sum_{i=0}^d \left| \prod_{j=0, j \neq i}^d \frac{x - z_j}{z_i - z_j} \right| = \\ &= (p-1) \sum_{i=0}^d |\mathcal{L}_i(x)|, \end{aligned} \quad (27)$$

where we call the polynomials $\{\mathcal{L}_i(x)\}_{i=0}^d$ as *Lagrange polynomials* or *Lagrange basis*. We want to obtain an upper-bound for the Lagrange-interpolation polynomial f^* on $[0, p)$ using Equation 27. Let us divide the $[0, p)$ interval into $2d$ equal length intervals $S_i := [\frac{p \cdot i}{2d}, \frac{p \cdot (i+1)}{2d})^{2d-1}$. For a better estimate on $\max_{x \in [0, p)} f^*(x)$, we slightly increase the running time of the extractor until we obtain sample points z_i that are sufficiently close to being equidistant. This helps us obtain a better upper bound on the Lagrange basis polynomials by avoiding “too small” denominators. Thus, the extractor selects uniformly at random interpolation points z_i from the interval $[0, p)$. According to the coupon collector’s problem [30], the extractor must sample $\approx 2d \log 2d$ points on average to sample at least one interpolation point from every segment in $S^* := \{S_i : i \equiv 0 \pmod{2}\}$. Let the set of interpolation points, $T = \{z_i\}_{i=0}^d$, consisting of the $d+1$ points chosen so that we choose one point from every segment in S^* . Note that points in T are close to be equidistant, i.e., $\forall i : \frac{p}{2d} \leq |z_i - z_{i+1}| \leq \frac{3p}{2d}$. We upper bound for these interpolation points $\max_{x \in [0, p)} f^*(x)$ as follows.

$$\begin{aligned} \max_{x \in [0, p)} f^*(x) &\leq (p-1) \sum_{i=0}^d \left| \prod_{j=0, j \neq i}^d \frac{x - z_j}{z_i - z_j} \right| \leq p \sum_{i=0}^d \left| \frac{\prod_{j=0, j \neq i}^d \frac{(j+1)p}{d}}{\frac{d/2}{\prod_{j=0, j \neq i}^d ((\frac{2j+1)p}{2d})^2}} \right| \leq \\ &\leq 2^{d+1} p \sum_{i=0}^d \left| \frac{\prod_{j=0, j \neq i}^d (j+1)}{\frac{d/2}{\prod_{j=0, j \neq i}^d (2j+1)^2}} \right| \leq 2^{d+1} p (d+1) \ll \alpha. \end{aligned}$$

Next, we show that f^* is p -faithful, i.e., p -faithfulness is implied by the following:

$$\max_{x \in [0, p)} f^*(x) \leq \alpha. \quad (28)$$

A successful evaluation of f^* at a point z means that $f^*(z) - k(z - \alpha) \in [0, \mathcal{B}]$ for some $k \in \mathbb{Z}$. Equation 28 implies that this can only happen for $k = 0$. Therefore, since $k = 0$ for every opened evaluation point, the extracted polynomial gives the correct mod p evaluation, i.e., $f^*(z)$. This concludes the proof of knowledge soundness as we showed that the extracted polynomial f^* behaves as f_0 from an evaluation point of view, more formally serves as an evaluation proxy for f_0 , cf. Definition 10. Additionally, the extracted polynomial f^* has the property of p -faithfulness (which f_0 may lack), i.e., opening the integer substitution value at a point z and reducing it mod p yields the same result as taking all coefficients of f^* mod p . ■

5.4 Polynomial binding

Lemma 6. *The Behemoth PC satisfies polynomial binding if the order assumption holds in the applied group of unknown order $\mathbb{G}$.*

Proof. Adversary $\mathcal{A}_1$ in its security game needs to compute the order of the GUO $\mathbb{G}$. To that end, $\mathcal{A}_1$ instantiates the Behemoth PC protocol in the GUO $\mathbb{G}$ and invokes an adversary $\mathcal{A}_0$ breaking the polynomial binding property of Behemoth to compute $|\mathbb{G}|$.

Assume towards contradiction a PPT adversary $\mathcal{A}_0$ that successfully outputs two polynomials $f_0, f_1 \in \mathbb{F}_p^{\leq d}[x]$ such that $f_0 \neq f_1$ and their commitment is the same. We create an efficient adversary $\mathcal{A}_1$ who breaks the order assumption, see Section 3.4 and [45]. Let us assume that $\mathcal{A}_0$ can open the commitment c to polynomials $f_0, f_1, f_0 \neq f_1$. Since $c = g^{\widehat{f_0(\alpha)}} = g^{\widehat{f_1(\alpha)}}$, therefore $g^{\widehat{f_0(\alpha)} - \widehat{f_1(\alpha)}} = \mathbf{1}$, i.e., $\widehat{f_0(\alpha)} - \widehat{f_1(\alpha)} = 0 \bmod |\mathbb{G}|$. Now, $\mathcal{A}_1$ outputs with non-negligible probability $\widehat{f_0(\alpha)} - \widehat{f_1(\alpha)}$ which is a multiple of the group order $|\mathbb{G}|$ whenever $\widehat{f_0(\alpha)} \neq \widehat{f_1(\alpha)}$ contradicting the order assumption.

Now, we deal with the case when $f_0(\alpha) = f_1(\alpha) \wedge f_0 \neq f_1$. Let $g := f_0 - f_1$ and the first non-zero monomial of g be x^k . Observe that $\sum_{i=0}^{k-1} (f_{0,i} - f_{1,i})\alpha^i \leq 2p \sum_{i=0}^{k-1} \alpha^i = \frac{2p(\alpha^k - 1)}{\alpha - 1} \leq \alpha^k$. The first inequality follows from $\forall i : |f_i| \leq p$, and the second inequality follows from $2p \ll \alpha$. Put differently, the first non-zero monomial dominates the rest of the sums of the remaining monomials. Hence, it cannot be the case for polynomials $f_0, f_1 \in \mathbb{F}_p^{\leq d}[x]$ that $\widehat{f_0(\alpha)} = \widehat{f_1(\alpha)} \wedge f_0 \neq f_1$. ■

5.5 Evaluation hiding

Lemma 7. *The Behemoth PC satisfies computational evaluation hiding if the discrete logarithm problem is hard in the group of unknown order $\mathbb{G}$.*

Proof. We can repeat the proof of evaluation hiding for the KZG polynomial commitment scheme. Suppose there exists an adversary $\mathcal{A}$ that breaks the evaluation hiding property of commitment c and correctly computes polynomial $f(x)$ (without loss of generality $\deg(f) = d$) given d valid witness tuples $(z_i, f(z_i), \pi_{\text{Open}, i})$. We show how to use $\mathcal{A}$ to construct an adversary $\mathcal{B}$ that can break the discrete logarithm assumption in $\mathbb{G}$. Let $(g, g^a) \in \mathbb{G} \times \mathbb{G}$ be a discrete logarithm instance that $\mathcal{B}$ needs to solve. $\mathcal{B}$ generates srs for $\mathcal{A}$ by appropriately picking $\alpha \in \mathbb{Z}$ and computing $\text{srs} = (\alpha, \{g^{\alpha^i}\}_{i=0}^d)$. $\mathcal{B}$ sets $(j, f(j)) \in_R \mathbb{Z}_p^2$ as polynomial $f(x)$'s evaluations at indices j . It then assumes

$f(0) = a$, which is the answer for the discrete logarithm instance, and computes $g^{f(\alpha)}$ using $d + 1$ exponentiated evaluations: $(0, g^a)$ and the d other chosen pairs $(j, g^{f(j)})$. Finally, $\mathcal{B}$ computes as part of the witnesses q_j for the d chosen evaluations $(j, f(j))$ as $q_j = (g^{f(\alpha)} / g^{f(j)})^{1/(\alpha-j)}$, and sends srs and d witness tuples $(j, f(j), \pi_{\text{Open},j})$ to $\mathcal{A}$. Once $\mathcal{A}$ returns polynomial $f(x)$, $\mathcal{B}$ returns the constant term $f(0)$ as the solution for the discrete logarithm instance. It is easy to see that the success probability of solving the discrete logarithm instance is the same as the success probability of $\mathcal{A}$, and the time required is a small constant larger than the time required by $\mathcal{A}$. ■

6 Transparent, Constant zkSNARKs

A well-known and popular recipe for devising efficient zkSNARKs for NP is to combine a secure **PC** scheme with a polynomial interactive oracle proofs (IOP) protocol [23, Theorem 4.] to obtain a zkSNARK. In the polynomial IOP paradigm, the prover sends oracles to polynomials that the verifier can query at random points [12]. After several rounds of communication, the verifier, having seen some evaluations of the received polynomial oracles, decides whether the claimed statement is true or not. For a formalization of the polynomial IOP paradigm, the reader is referred to [12, 23]. The polynomial oracles sent by the prover to the verifier can be instantiated with polynomial commitments. Polynomial IOPs and polynomial commitment schemes offer a vast design and trade-off space, allowing practitioners to choose the characteristics (e.g., trust assumptions, prover and verifier efficiency, proof size, etc.) that best suit their applications. At the time of writing, the state-of-the-art polynomial IOP is the PLONK polynomial IOP by Gabizon, Williamson, and Ciobotaru [33]. Plonk is a 3-round honest verifier zero-knowledge polynomial IOP with preprocessing for any NP statement $\mathcal{R}$ with arithmetic complexity n that makes 12 queries to 12 univariate degree n polynomial oracles. The total number of distinct query points is 2. The preprocessing verifier does $\mathcal{O}(n)$ work to check 7 of the univariate degree n polynomials.

When the Plonk polynomial IOP is compiled with the Behemoth **PC** scheme, it yields the first transparent zkSNARK with *constant communication and verifier complexity* in the polynomial-IOP paradigm.

Theorem 3. (*Transparent, Constant SNARK.*) *There exists an $\mathcal{O}(1)$ -round public coin interactive argument of knowledge for any NP relation of arithmetic complexity n that has $\mathcal{O}(1)$ communication, $\mathcal{O}(1)$ online verification, cubic prover time, and a preprocessing step that is verifiable in quasilinear time. The argument of knowledge has knowledge soundness, assuming it is instantiated with a group $\mathbb{G}$ of unknown order in which the strong RSA assumption and the adaptive root assumption hold.*

We note that transparent, succinct arguments with the same asymptotics exist outside the polynomial-IOP paradigm due to Lai and Malavolta [42]. They apply subvector commitments and probabilistically checkable proofs (PCP) to achieve constant-size, transparent arguments with constant-time verifiers.

7 Performance Analysis

This section studies the theoretical and practical performance of our **PC** scheme.

7.1 Transparent Setup Efficiency

The $\text{GenSRS}(1^\lambda, d)$ algorithm is transparent and somewhat time-sensitive as it incurs large exponentiations; computing g^{α^d} entails $d \log \alpha$ repeated squaring. Since $\alpha \approx p^d$, the setup algorithm has a quadratic complexity $\mathcal{O}(d^2 \log p)$ in the degree d of the committed polynomial. This transparent setup only needs to be performed once. It is extensible in the sense that there is no limitation on the maximum degree of the committed polynomial. The extensibility of the srs might be valuable in certain applications, e.g., zkSNARKs. Whoever computes the Behemoth srs can also prove the correctness of the setup by proving that for every pair of neighboring elements in the srs, the exponentiation was done correctly. They can prove this by enclosing proofs of exponentiations for $\forall i : (g^{\alpha^i}, g^{\alpha^{i+1}})_{i=0}^{d-1}$ [48, 58]. In a typical parameters setting ($d = 2^{20}, p \approx 2^{256}$), the setup algorithm entails computing g^{α^d} , i.e., one must compute $\approx d^2 \log p = 2^{48}$ repeated squarings to complete the transparent setup. This computation has a similar computational complexity to the LCS time-lock puzzle created by Rivest [50]. To compute the LCS time-lock puzzle, one needed to compute $\approx 2^{47}$ repeated squarings. Originally, the LCS time-lock puzzle was intended to last 35 years. However, on specialized hardware using novel techniques [44], it is possible to accomplish this computation in less than two months. Certainly, the complexity of the setup algorithm becomes more feasible for smaller polynomial commitment schemes, e.g., $d \in \{2^{12}, 2^{13}, 2^{14}, 2^{15}\}$. Such a shorter **PC** scheme has been recently deployed for a data availability application on the Ethereum blockchain [59]. We evaluate the practical performance cost of completing the Behemoth transparent setup in RSA and class groups for various parameter settings when using the currently available best, specialized hardware implementations of the corresponding group operations [44, 60], cf. Table 1. Our transparent setup for larger committed polynomials, i.e., $d \approx 2^{25} - 2^{30}$ becomes practically infeasible to complete when the polynomial ring is defined over a prime with 256 bits.

Degree d	2^{10}	2^{12}	2^{13}	2^{14}	2^{15}	2^{20}	2^{25}	2^{30}
Group $\mathbb{G}$								
RSA 2048-bits	0.54 secs	8.59 secs	34.36 secs	2.291 mins	9.163 mins	6.516 days	18.27 years	18706 years
Class 2048-bits	31.8 mins	8.47 hrs	1.41 days	5.65 days	22.6 days	63.3 years	64800 years	$6.64 \cdot 10^7$ years

Table 1: We provide time estimates for completing the transparent setup for a given GUO $\mathbb{G}$ and maximum supported degree d of the committed polynomial. For every cell, we consider the size of the base field of the polynomial ring $\mathbb{F}_p^{\leq d}[x]$ to be a 256-bit prime, as it is the case in most applications. The state of the art is that on specialized hardware, i.e., ASICs, one iteration of the function $f(t) : t \rightarrow g^{2^t}$ takes 2ns for a 2048-bit RSA group. In the case of class groups with 2048-bit discriminant, we have that one iteration of the VDF function takes 7.1 μ s [60].

	Com	Open proof sizes	Open time complexity		srs	Setup
		$ \pi_{\text{Open}} $	Prove	Verify		
KZG [37]	$1\mathbb{G}_P$	$1\mathbb{G}_P$	$\mathcal{O}(d)$	$\mathcal{O}(1)$	$\mathcal{O}(d)\mathbb{G}_P$	Trusted
Bootle et al. [19]	$1\mathbb{G}$	$\mathcal{O}(\sqrt{d})\mathbb{G}$	$\mathcal{O}(d)$	$\mathcal{O}(\sqrt{d})$	$\mathcal{O}(\sqrt{d})\mathbb{G}$	Transparent
Bulletproofs [22]	$1\mathbb{G}$	$2 \log d \mathbb{G}$	$\mathcal{O}(d)$	$\mathcal{O}(d)$	$\mathcal{O}(d)\mathbb{G}$	Transparent
FRI [10]	$1\mathbb{G}$	$\lambda \log^2 d \mathbb{G}$	$\mathcal{O}(d \log d)$	$\mathcal{O}(\lambda \log^2 d)$	$\mathcal{O}(1)\mathbb{G}$	Transparent
DARK [23]	$1\mathbb{G}_U$	$2 \log d \mathbb{G}_U + 2 \log d \mathbb{F}_p$	$\mathcal{O}(d \log d)$	$\mathcal{O}(\log d)$	$\mathcal{O}(1)\mathbb{G}_U$	Transparent
Dory [43]	$1\mathbb{G}_P$	$6 \log d \mathbb{G}_P$	$\mathcal{O}(d^{\log 8 / \log 25})$	$\mathcal{O}(\log d)$	$\mathcal{O}(d)\mathbb{G}_P$	Transparent
Dew [4]	$1\mathbb{G}_U$	$66\mathbb{G}_U$	$\mathcal{O}(d^3 / \log d)$	$\mathcal{O}(\log d)$	$\mathcal{O}(1)\mathbb{G}_U$	Transparent
Behemoth (this work)	$1\mathbb{G}_U$	$47\mathbb{G}_U + 19\mathbb{F}_p$	$\mathcal{O}(d^3 / \log d)$	$\mathcal{O}(1)$	$\mathcal{O}(d)\mathbb{G}_U$	Transparent

Table 2: Comparing the theoretical performances of polynomial commitment schemes. We only enclose the most efficient representative from each cryptographic approach to keep the table compact to obtain **PC** schemes. Properties in **red** are undesirable or impractical, and properties in **orange** become an issue for polynomials with larger degrees d . Properties achieved in **green** indicate practical, efficient constructions or desirable characteristics. To account for the differences in the concrete efficiency of the applied groups, we denote the applied cryptographic groups differently. Specifically, schemes that are instantiated in groups equipped with bilinear pairings are denoted as $\mathbb{G}_P$, while groups of unknown orders are denoted as $\mathbb{G}_U$. Groups where one only needs to assume the discrete logarithm assumption or the existence of one-way functions are denoted as $\mathbb{G}$. Here, we report an optimized Behemoth proof size achieved by proof batching described in Section 7. Recall that Dew’s verifier complexity is $\mathcal{O}(\log d)$ *field operations*, i.e., for practical parameter choices, logarithmic verifier complexity might be better than $\mathcal{O}(1)$ verifier complexity.

7.2 Prover Efficiency

Committing to a polynomial f can be done by computing $g^{\widehat{f(\alpha)}}$. Though the size of $\widehat{f(\alpha)}$ is huge over the integers ($\approx p^{d+1}$), the prover can compute this exponentiation in $\mathcal{O}(d)$ time, since all the monomials g^{α^i} of α are provided in the srs. Hence, the prover only needs to compute d small exponentiations with exponents of length $\log p$.

Computing the opening proof for the statement $f(z) = s$ is more computationally heavy. First, the prover needs to compute $g^{\widehat{q(\alpha)}}$, where the quotient polynomial $q(x)$ is defined as $q(x) := \frac{f(x) - \widehat{s}}{x - z}$. The prover computes directly this polynomial division using Horner’s method. It computes $g^{\widehat{q(\alpha)}}$ on a rolling basis, i.e., monomial by monomial, using the algorithm detailed in Figure 4. This strategy leads to an $\mathcal{O}(d^2)$ computation. Completing the opening proof (both to compute the ProveEvaluation and the PoKDegUp proofs) requires the computation of the following NIZKs.

π_{PoE} : using the techniques of Wesolowski [58], a PoE proof consists of a single group element, see Appendix A.2. The biggest chunk of the prover’s work in this NIZK is incurred by computing a large modular division: $x = ql + r$, where $x \approx p^{d+1}$ and $l, r \approx \lambda$. Computing $q \in \mathbb{Z}$ takes quasi linear time in the number of digits of x , i.e., $\mathcal{O}(d \log d \log p)$.

Polynomial division algorithm for the ProveEvaluation protocol.

Input: $(\text{srs}, f(x) \in \mathbb{F}_p[x], z, s = f(z))$. $\mathcal{P}$ wants to compute $q(\alpha) = \frac{f(\alpha) - \hat{s}}{\alpha - z}$ in the exponent.

1. Let $Q := g^0$.
2. $\forall i \in [0, d]$ do the following:
 - Compute q_{d-i} , i.e., the $(d-i)$ th coefficient of $\frac{f(x) - \hat{s}}{x - z}$. Let $Q := Q \cdot (g^{\alpha^{d-i}})^{q_{d-i}}$.

Output: $Q = g^{q(\alpha)}$.

Fig. 4: Polynomial division algorithm to compute $q(x) = \frac{f(x) - \hat{s}}{x - z}$, and $g^{q(\alpha)}$ in the exponent as part of the ProveEvaluation protocol. We note here that $\log_2(q_{d-i}) \approx i \log_2(p)$, hence, this is a quadratic algorithm in the degree of the committed polynomial.

π_{PoKEMon} : a PoKEMon proof consists of a single group element and a small integer r with size $\approx p$, see Appendix A.3. Also, in this case, the prover's work is quasi-linear, i.e., $\mathcal{O}(d \log d \log p)$.

π_{PoKPE} : Arguing about the positivity of $\hat{s}$ and $\mathcal{B} - \hat{s}$ for $\hat{s} = f(z)$ requires the prover to find three (four) squares that sum up to $\hat{s}$ and $\mathcal{B} - \hat{s}$, respectively. The size of both of these integers $\hat{s}, \mathcal{B} - \hat{s}$ is roughly $(d+1) \log p$ bits. The state-of-the-art algorithm by Pollack and Treviño finds the (three) four squares decomposition of an integer n in time $\mathcal{O}(\log^2 n / \log \log n)$. Hence, creating the PoKPE proof takes approximately $\mathcal{O}(d^2 \log^2 p / (\log d + \log \log p))$ arithmetic operations, i.e., quasi quadratic in the degree of the committed polynomial. Since the algorithm operates on integers of length $d \log p$ bits, in practice, the computational complexity of creating the PoKPE proof via the three square decompositions is cubic. The proof consists of 6 group elements and 3 small ($\approx p$) integers.

π_{PoKE} : this proof system has the same complexity as the PoE proof that is $\mathcal{O}(d \log d \log p)$, though the proof consists of a group element from $\mathbb{G}$ and an integer in $\mathbb{F}_p$.

π_{ChaumP} : the Chaum-Pedersen DLEq proof has the same complexity as generating two PoKE proofs. It has a proof size of two group elements from $\mathbb{G}$ and two small integers from $\mathbb{F}_p$.

The ProveEvaluation protocol consists of 1 PoE, 2 PoKEMon, and 2 PoKPE proofs. The PoKDdegUp proof consists of 3 PoKE, 1 Chaum-Pedersen DLEq, and 1 ProveEvaluation proof. Altogether the Open protocol requires the computation of 4 PoE, 8 PoKEMon, 8 PoKPE, 6 PoKE, and 2 Chaum-Pedersen DLEq proofs. Therefore, the unoptimized Behemoth Open proof size consists of $70\mathbb{G}$ and $42\mathbb{F}_p$ elements. Next, we mention techniques to shrink the proof size even further.

Batching the applied NIZK proofs. The Open protocol applies 24 instances of the PoKE, PoKPE, ChaumP, PoKEMon proofs where the verification equation checks that the prover knows a random prime x_i -root w_i of a public element a_i , i.e., $w_i^{x_i} = a_i$ for $i \in [0, 24]$. We have that with overwhelming probability for the λ -bit challenge

primes: $\forall i, j (i \neq j) : \gcd(x_i, x_j) = 1$. That allows us to make the opening proof even more succinct by batching the underlying NIZK proofs due to the protocol PoKCR (aggregating knowledge of co-prime roots) introduced in [16], see also Appendix A.6.

7.3 Verifier Efficiency

The verifier runs in constant time. Verifying an opening proof entails verifying several NIZKs as subprotocols, i.e., PoE, PoKEMon, PoKPE, PoKE, ChaumP, all requiring constant time. The verifier needs to compute a constant number of group operations to verify a Behemoth Open proof. In particular, an unoptimized (without proof batching) version of the Open protocol requires the computation of 210 group operations.

	$ \pi $	P's concrete complexity		V's concrete complexity	
		$\mathbb{G}$	$\mathbb{F}_p$	$\mathbb{G}$	$\mathbb{F}_p$
PoE($Q, \alpha - z$) ^a [58]	$1\mathbb{G}$	$\mathcal{O}(d \log p)$	$\mathcal{O}(d \log d \log p)$	3	0
PoKEMon [16]	$1\mathbb{G} + 1\mathbb{F}_p$	$\mathcal{O}(d \log p)$	$\mathcal{O}(d \log d \log p)$	3	0
PoKE [16]	$1\mathbb{G} + 1\mathbb{F}_p$	$\mathcal{O}(d \log p)$	$\mathcal{O}(d \log d \log p)$	3	0
ChaumP (DLEq) [16, 24]	$2\mathbb{G} + 2\mathbb{F}_p$	$\mathcal{O}(d \log p)$	$\mathcal{O}(d \log d \log p)$	6	0
PoKPE [4]	$6\mathbb{G} + 3\mathbb{F}_p$	$\mathcal{O}(d \log p)$	$\mathcal{O}(d^3 \log^3 p / (\log d + \log \log p))$	24	0
ProveEvaluation (cf. Section 4.3)	$15\mathbb{G} + 8\mathbb{F}_p$	$\mathcal{O}(d \log p)$	$\tilde{\mathcal{O}}(d^3 / \log d)$	33	0
PoKDegUp (cf. Section 4.3)	$20\mathbb{G} + 13\mathbb{F}_p$	$\mathcal{O}(d \log p)$	$\tilde{\mathcal{O}}(d^3 / \log d)$	72	0
Open (cf. Section 4.2)	$70\mathbb{G} + 42\mathbb{F}_p$	$\mathcal{O}(d \log p)$	$\tilde{\mathcal{O}}(d^3 / \log d)$	210	0

Table 3: Behemoth Open protocol's proof sizes and concrete computational costs for the prover and verifier, respectively. The Open protocol consists of several subprotocols. For each subprotocol, we enlist the proof size and the number of group operations the prover and the verifier needs to compute in the applied group of unknown order $\mathbb{G}$ and the base field $\mathbb{F}_p$ of the polynomial ring $\mathbb{F}_p^{\leq d}[x]$. The Open protocol consists of two executions of the ProveEvaluation and the PoKDegUp protocols. This table considers an unoptimized version of the Behemoth Open protocol, i.e., without proof batching.

^a Note that in our variant of Wesolowski's PoE protocol, the verifier outsources its computation in $\mathbb{F}_p$ to the prover. Hence, the verifier does not need to compute group operations in $\mathbb{F}_p$.

8 Conclusion and Open Problems

In this work, to the best of our knowledge, we constructed the first transparent polynomial commitment scheme that achieves both constant-size opening proofs and verification time. The main idea of our construction is to instantiate the KZG opening strategy in a group of unknown order. Hence, our construction affirmatively answers the question of succinct, transparent polynomial commitment schemes raised in [46]. However, several challenging open problems and research directions remain.

8.1 Prover efficiency

The downside of our construction is the increased prover cost. Ideally, one wants to achieve a (quasi)-linear prover time that is also concretely efficient. Unfortunately, cubic prover time is concretely impractical in most applications. Hence, making our prover asymptotically and concretely more efficient would be fascinating. One of the bottlenecks of our construction is finding the four-square decomposition of an integer. Is there a concretely efficient algorithm that finds the four-square decomposition of an integer in quasi-linear time (in the integer's bit length)?

8.2 Batching opening proofs and other extensions

The KZG PCS offers batching capabilities for opening proofs. Batching opening proofs for the same polynomial was already introduced in the KZG paper. Boneh et al. introduce an extension of the KZG scheme [17], where one can batch opening proofs for multiple points opened at multiple polynomials. Feist and Khovratovich design a protocol that allows the fast computation of KZG opening proofs where primitive roots are the opened points [31]. It seems accessible to adapt all these techniques to the group of unknown order setting. We leave the security and efficiency analysis of these protocols in the Behemoth setting as future work. Another fruitful direction of future work might be to extend Behemoth commitments to multivariate commitments akin to Papamanthou et al. [47], who extended the KZG **PC** scheme to multivariate polynomials at the expense of increased srs, i.e., quadratic, $\binom{d}{2}$, in the case of bivariate polynomials, and $\binom{d}{k}$ for k -variate polynomials.

8.3 Succinct, transparent, post-quantum polynomial commitment schemes

Existing **PC** schemes with constant-size evaluation proofs and verifiers are not post-quantum secure. The ultimate **PC** scheme would possess these beneficial performance characteristics *and post-quantum security* as well. Therefore, it is an interesting research direction to design post-quantum secure, transparent polynomial commitment schemes [9, 34] with *both constant evaluation proofs and constant verifier*. The currently known most efficient, post-quantum secure **PC** schemes [2, 8, 20] apply lattice-based cryptographic assumptions. At the time of writing, there is no known post-quantum secure, transparent polynomial commitment scheme with constant-size proofs and constant-time verifier. Therefore, we conclude with an exciting open question:

Is there a transparent, plausibly post-quantum secure polynomial commitment scheme that achieves constant-size opening proofs and constant-time verifiers?

We leave the design of such a **PC** scheme or an impossibility result to future work.

Acknowledgements

This research was supported by the Ministry of Culture and Innovation and the National Research, Development, and Innovation Office within the Quantum Information National Laboratory of Hungary (Grant No. 2022-2.1.1-NL-2022-00004).

References

1. Aggarwal, D., Maurer, U.: Breaking rsa generically is equivalent to factoring. *IEEE Transactions on Information Theory* **62**(11), 6251–6259 (2016) [6](#)
2. Albrecht, M.R., Fenzi, G., Lapiha, O., Nguyen, N.K.: Slap: succinct lattice-based polynomial commitments from standard assumptions. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. pp. 90–119. Springer (2024) [30](#)
3. Ames, S., Hazay, C., Ishai, Y., Venkitasubramaniam, M.: Liger: Lightweight sublinear arguments without a trusted setup. In: *Proceedings of the 2017 acm sigsac conference on computer and communications security*. pp. 2087–2104 (2017) [2](#)
4. Arun, A., Ganesh, C., Lokam, S., Mopuri, T., Sridhar, S.: Dew: Transparent constant-sized zksnarks. *Cryptology ePrint Archive* (2022) [2](#), [9](#), [14](#), [20](#), [27](#), [29](#), [35](#), [36](#)
5. van Baarsen, A.: Imaginary quadratic class groups and a survey of time-lock cryptographic applications (2023) [13](#), [20](#)
6. van Baarsen, A., Stevens, M.: On time-lock cryptographic assumptions in abelian hidden-order groups. In: *Advances in Cryptology–ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part II* 27. pp. 367–397. Springer (2021) [7](#), [19](#), [20](#)
7. Barić, N., Pfizmann, B.: Collision-free accumulators and fail-stop signature schemes without trees. In: *International conference on the theory and applications of cryptographic techniques*. pp. 480–494. Springer (1997) [7](#)
8. Baum, C., Bootle, J., Cerulli, A., Del Pino, R., Groth, J., Lyubashevsky, V.: Sub-linear lattice-based zero-knowledge arguments for arithmetic circuits. In: *Advances in Cryptology–CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part II*. pp. 669–699. Springer (2018) [30](#)
9. Belling, A., Soleimani, A.: Vortex: Building a lattice-based snark scheme with transparent setup. *Cryptology ePrint Archive* (2022) [30](#)
10. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Fast reed-solomon interactive oracle proofs of proximity. In: *45th international colloquium on automata, languages, and programming (icalp 2018)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik (2018) [2](#), [6](#), [27](#)
11. Ben-Sasson, E., Chiesa, A., Green, M., Tromer, E., Virza, M.: Secure sampling of public parameters for succinct zero knowledge proofs. In: *2015 IEEE Symposium on Security and Privacy*. pp. 287–304. IEEE (2015) [2](#)
12. Ben-Sasson, E., Chiesa, A., Spooner, N.: Interactive oracle proofs. In: *Theory of Cryptography Conference*. pp. 31–60. Springer (2016) [1](#), [25](#)
13. Block, A.R., Holmgren, J., Rosen, A., Rothblum, R.D., Soni, P.: Time- and space-efficient arguments from groups of unknown order. In: *Advances in Cryptology–CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part IV* 41. pp. 123–152. Springer (2021) [2](#)
14. Blum, M., Feldman, P., Micali, S.: Non-interactive zero-knowledge and its applications. In: *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*, pp. 329–349. ACM (2019) [8](#)

15. Boneh, D., Bonneau, J., Bünz, B., Fisch, B.: Verifiable delay functions. In: Annual international cryptology conference. pp. 757–788. Springer (2018) [12](#)
16. Boneh, D., Bünz, B., Fisch, B.: Batching techniques for accumulators with applications to iops and stateless blockchains. In: Annual International Cryptology Conference. pp. 561–586. Springer (2019) [6](#), [7](#), [8](#), [9](#), [14](#), [19](#), [20](#), [29](#), [34](#), [35](#), [37](#)
17. Boneh, D., Drake, J., Fisch, B., Gabizon, A.: Efficient polynomial commitment schemes for multiple points and polynomials. Cryptology ePrint Archive (2020) [1](#), [30](#)
18. Boneh, D., Drake, J., Fisch, B., Gabizon, A.: Halo infinite: Proof-carrying data from additive polynomial commitments. In: Annual International Cryptology Conference. pp. 649–680. Springer (2021) [5](#)
19. Bootle, J., Cerulli, A., Chaidos, P., Groth, J., Petit, C.: Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 327–357. Springer (2016) [2](#), [27](#)
20. Bootle, J., Lyubashevsky, V., Nguyen, N.K., Seiler, G.: A non-pcp approach to succinct quantum-safe zero-knowledge. In: Advances in Cryptology–CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part II. pp. 441–469. Springer (2020) [30](#)
21. Bowe, S., Gabizon, A., Miers, I.: Scalable multi-party computation for zk-snark parameters in the random beacon model. Cryptology ePrint Archive (2017) [2](#)
22. Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G.: Bulletproofs: Short proofs for confidential transactions and more. In: 2018 IEEE symposium on security and privacy (SP). pp. 315–334. IEEE (2018) [2](#), [27](#)
23. Bünz, B., Fisch, B., Szepieniec, A.: Transparent snarks from dark compilers. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 677–706. Springer (2020) [2](#), [5](#), [25](#), [27](#)
24. Chaum, D., Pedersen, T.P.: Wallet databases with observers. In: Annual international cryptology conference. pp. 89–105. Springer (1992) [8](#), [16](#), [29](#), [34](#)
25. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, N., Ward, N.: Marlin: preprocessing zksnarks with universal and updatable srs. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 738–768. Springer (2020) [1](#)
26. Chu, H., Fiore, D., Kolonelos, D., Schröder, D.: Inner product functional commitments with constant-size public parameters and openings. Cryptology ePrint Archive (2022) [2](#)
27. Damgård, I., Koprowski, M.: Generic lower bounds for root extraction and signature schemes in general groups. In: International Conference on the Theory and Applications of Cryptographic Techniques. pp. 256–271. Springer (2002) [6](#)
28. Das, S., Xiang, Z., Ren, L.: Powers of tau in asynchrony. Cryptology ePrint Archive (2022) [2](#)
29. Dobson, S., Galbraith, S.D., Smith, B.: Trustless unknown-order groups. Cryptology ePrint Archive (2020) [2](#), [11](#)
30. Erdős, P., Rényi, A.: On a classical problem of probability theory. Magyar Tud. Akad. Mat. Kutató Int. Közl **6**(1), 215–220 (1961) [23](#)
31. Feist, D., Khovratovich, D.: Fast amortized kzg proofs. Cryptology ePrint Archive (2023) [1](#), [30](#)
32. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Conference on the theory and application of cryptographic techniques. pp. 186–194. Springer (1986) [9](#), [19](#)
33. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive (2019) [25](#)
34. Golovnev, A., Lee, J., Setty, S., Thaler, J., Wahby, R.S.: Brakedown: Linear-time and post-quantum snarks for r1cs. Cryptology ePrint Archive (2021) [30](#)
35. Hafner, J.L., McCurley, K.S.: A rigorous subexponential algorithm for computation of class groups. Journal of the American mathematical society **2**(4), 837–850 (1989) [2](#)

36. Jager, T., Schwenk, J.: On the analysis of cryptographic assumptions in the generic ring model. *Journal of cryptology* **26**, 225–245 (2013) [6](#)
37. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: *International conference on the theory and application of cryptology and information security*. pp. 177–194. Springer (2010) [1](#), [27](#)
38. Kattis, A., Panarin, K., Vlasov, A.: Redshift: transparent snarks from list polynomial commitment iops. *Cryptology ePrint Archive* (2019) [2](#)
39. Kohlweiss, M., Maller, M., Siim, J., Volkhov, M.: Snarky ceremonies. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 98–127. Springer (2021) [2](#)
40. Kuszmaul, J.: Verkle trees. *Verkle Trees* **1** (2019) [1](#)
41. Lagarias, J.C.: Worst-case complexity bounds for algorithms in the theory of integral quadratic forms. *Journal of Algorithms* **1**(2), 142–186 (1980) [13](#), [16](#)
42. Lai, R.W., Malavolta, G.: Subvector commitments with application to succinct arguments. In: *Advances in Cryptology—CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part I* 39. pp. 530–560. Springer (2019) [25](#)
43. Lee, J.: Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In: *Theory of Cryptography Conference*. pp. 1–34. Springer (2021) [2](#), [27](#)
44. Mert, A.C., Ozturk, E., Savas, E.: Low-latency asic algorithms of modular squaring of large integers for vdf evaluation. *IEEE Transactions on Computers* (2020) [26](#)
45. Miller, G.L.: Riemann’s hypothesis and tests for primality. *Journal of computer and system sciences* **13**(3), 300–317 (1976) [7](#), [24](#)
46. Nikolaenko, V., Ragsdale, S., Bonneau, J., Boneh, D.: Powers-of-tau to the people: Decentralizing setup ceremonies. *Cryptology ePrint Archive* (2022) [2](#), [29](#)
47. Papamanthou, C., Shi, E., Tamassia, R.: Signatures of correct computation. In: *Theory of Cryptography Conference*. pp. 222–242. Springer (2013) [30](#)
48. Pietrzak, K.: Simple verifiable delay functions. In: *10th innovations in theoretical computer science conference (itsc 2019)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik (2018) [8](#), [9](#), [12](#), [20](#), [26](#)
49. Pollack, P., Treviño, E.: Finding the four squares in lagrange’s theorem. *Integers* **18**, A15 (2018) [12](#), [20](#), [36](#)
50. Rivest, R.L.: Description of the lcs35 time capsule crypto-puzzle. Retrieved at<> Published on: Apr **4**, 6 (1999) [26](#)
51. Rotem, L., Segev, G.: Generically speeding-up repeated squaring is equivalent to factoring: Sharp thresholds for all generic-ring delay functions. In: *Annual International Cryptology Conference*. pp. 481–509. Springer (2020) [6](#)
52. Shoup, V.: Lower bounds for discrete logarithms and related problems. In: *Advances in Cryptology—EUROCRYPT’97: International Conference on the Theory and Application of Cryptographic Techniques Konstanz, Germany, May 11–15, 1997 Proceedings* 16. pp. 256–266. Springer (1997) [6](#), [7](#), [34](#)
53. Sutherland, A.V.: Order computations in generic groups. Ph.D. thesis, Massachusetts Institute of Technology (2007) [2](#)
54. Thakur, S.: Batching non-membership proofs with bilinear accumulators. *Cryptology ePrint Archive* (2019) [16](#)
55. Thakur, S.: Arguments of knowledge via hidden order groups. *Cryptology ePrint Archive* (2020) [8](#), [34](#)
56. Thakur, S.: Constructing hidden order groups using genus three jacobians. *Cryptology ePrint Archive* (2020) [2](#)

57. Wahby, R.S., Tzialla, I., Shelat, A., Thaler, J., Walfish, M.: Doubly-efficient zkSNARKs without trusted setup. In: 2018 IEEE Symposium on Security and Privacy (SP). pp. 926–943. IEEE (2018) [2](#)
58. Wesolowski, B.: Efficient verifiable delay functions. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 379–407. Springer (2019) [7](#), [8](#), [9](#), [12](#), [14](#), [20](#), [26](#), [27](#), [29](#), [34](#)
59. Wood, G., et al.: Ethereum: A secure decentralised generalised transaction ledger. Ethereum project yellow paper **151**(2014), 1–32 (2014) [26](#)
60. Zhu, D., Tian, J., Li, M., Wang, Z.: Low-latency hardware architecture for vdf evaluation in class groups. IEEE Transactions on Computers (2022) [26](#)

A The applied NIZK proof systems

In this paper, we rely on non-interactive zero-knowledge proofs built for the following efficiently decidable languages. The following proof systems were proven secure in the generic group model [52] by Boneh, Bünz, and Fisch [16], and Thakur [55].

A.1 Chaum-Pedersen proof for discrete logarithm equality (ChaumP)

Chaum and Pedersen [24] introduced an efficient proof system for the following relation.

$$\mathcal{R}_{\text{ChaumP}} = \{((s, t, u, v \in \mathbb{G}); x \in \mathbb{Z}) : s = t^x \wedge u = v^x\}. \quad (29)$$

Their protocol is not directly applicable in our case, as we work in a group of unknown order, while the original ChaumP protocol was introduced for cyclic groups with known prime order. However, Thakur showed in [55] how to adapt the original Chaum-Pedersen discrete logarithm equality (DLEq) protocol to the group of unknown order setting.

Params: $\mathbb{G} \xleftarrow{\$} G\text{Gen}(\lambda), g \in \mathbb{G}$. Inputs: $s, t, u, v \in \mathbb{G}, x \in \mathbb{Z}$. Claim: $t^x = s \wedge v^x = u$.

1. $\mathcal{P}$ sends $\hat{g} := g^x$ to $\mathcal{V}$.
2. $\mathcal{V}$ sends $l \in_R \text{Primes}(\lambda)$ to $\mathcal{P}$.
3. $\mathcal{P}$ computes $q = \lfloor \frac{x}{l} \rfloor \in \mathbb{Z} \wedge r \in [l]$, where $x = ql + r$. $\mathcal{P}$ also computes $Q_1 = t^q \in \mathbb{G}, Q_2 = v^q \in \mathbb{G}, g^* := g^q$ and sends $(Q_1, Q_2, g^*, r) \in \mathbb{G}^3 \times [l]$ to $\mathcal{V}$.
4. $\mathcal{V}$ checks $r \in [l]$ and verifies the equations $s \stackrel{?}{=} Q_1^l t^r \wedge u \stackrel{?}{=} Q_2^l v^r \wedge (g^*)^l g^r \stackrel{?}{=} \hat{g}$.

Fig. 5: The ChaumP discrete logarithm equality protocol for groups of unknown order.

A.2 Proof of Exponentiation (PoE)

Wesolowski introduced a constant-size proof system for the following language [58]:

$$\mathcal{R}_{\text{PoE}} = \{((u, w \in \mathbb{G}), x \in \mathbb{Z}; \perp) : w = u^x \in \mathbb{G}\}. \quad (30)$$

Note there is no secret witness in the language $\mathcal{R}_{\text{PoE}}$. The following protocol yields a proof for $\mathcal{R}_{\text{PoE}}$ that consists of one group element. The verifier's work consists of two group operations in $\mathbb{G}$ and the computation of $q = \lfloor \frac{x}{l} \rfloor \in \mathbb{Z}$. In our application $x \approx \alpha \approx p^d$. Since computing q takes $\mathcal{O}(\log d)$ steps and we want a constant verifier, therefore, we outsource this computation to the prover with the help of the following proof system.

Params: $\mathbb{G} \xleftarrow{\$} GGen(\lambda)$. Inputs: $u, w \in \mathbb{G}, x \in \mathbb{Z}$. Claim: $u^x = w$.

1. $\mathcal{V}$ sends $l \in_R \text{Primes}(\lambda)$ to $\mathcal{P}$.
2. $\mathcal{P}$ computes $q = \lfloor \frac{x}{l} \rfloor \in \mathbb{Z} \wedge r \in [l]$, where $x = ql + r$. $\mathcal{P}$ sends $Q = u^q \in \mathbb{G}$ to $\mathcal{V}$.
3. $\mathcal{V}$ computes $r = x \bmod l$ and checks $w \stackrel{?}{=} Q^l u^r$.

Fig. 6: The PoE protocol.

A.3 Proof of Knowledge of Exponent Modulo an odd integer (PoKEMon)

An efficient proof system for the following useful language in GUOs was given in [16].

$$\mathcal{R}_{\text{PoKEMon}} = \{((w \in \mathbb{G}, \hat{x} \in [n]); x \in \mathbb{Z}) : w = g^x \in \mathbb{G}, x \bmod n = \hat{x}\}. \quad (31)$$

Params: $\mathbb{G} \xleftarrow{\$} GGen(\lambda), g \in \mathbb{G}$. Inputs: Odd prime $n, w \in \mathbb{G}, \hat{x} \in [n]$. Witness: $x \in \mathbb{Z}$; Claim: $g^x = w \wedge x \equiv \hat{x} \bmod n$.

1. $\mathcal{V}$ sends $l \in_R \text{Primes}(\lambda)$ to $\mathcal{P}$.
2. $\mathcal{P}$ computes $q \in \mathbb{Z} \wedge r \in [l \cdot n]$, where $x = q(l \cdot n) + r$. $\mathcal{P}$ sends $Q = g^q \in \mathbb{G}$ and r to $\mathcal{V}$, i.e., $\pi = (Q, r)$.
3. $\mathcal{V}$ accepts if $r \in [l \cdot n] \wedge r \equiv \hat{x} \bmod n \wedge w \stackrel{?}{=} Q^{l \cdot n} g^r$.

Fig. 7: The PoKEMon protocol.

A.4 Proof of Knowledge of Squared Exponent (PoKSE)

For the following important language, a useful proof system was devised in [4, 16].

$$\mathcal{R}_{\text{PoKSE}} = \{((w \in \mathbb{G}); x \in \mathbb{Z}) : w = g^{x^2} \in \mathbb{G}\}. \quad (32)$$

Params: $\mathbb{G} \xleftarrow{\$} GGen(\lambda), g \in_R \mathbb{G}$. Inputs: $w \in \mathbb{G}$. Witness: $x \in \mathbb{Z}$; Claim: $g^{x^2} = w$.

1. $\mathcal{P}$ sends $z = g^x$ to $\mathcal{V}$.
2. $\mathcal{V}$ sends $l \in_R \text{Primes}(\lambda)$ to $\mathcal{P}$.
3. $\mathcal{P}$ computes $q \in \mathbb{Z} \wedge r \in [l]$, where $x = ql + r$. $\mathcal{P}$ sends $Q = z^q, Q' = g^q \in \mathbb{G}$ and r to $\mathcal{V}$, i.e., $\pi = (Q, Q', r)$.
4. $\mathcal{V}$ accepts if $r \in [l] \wedge w \stackrel{?}{=} Q^l z^r \wedge z \stackrel{?}{=} Q'^l g^r$.

Fig. 8: The PoKSE protocol.

A.5 Proof of Knowledge of Positive Exponent (PoKPE)

The following proof system allows one to prove with a constant-size proof that a committed integer is non-negative in a group of unknown order [4]. We note that this essentially yields an efficient range proof.

$$\mathcal{R}_{\text{PoKPE}} = \{((w \in \mathbb{G}); x \in \mathbb{Z}) : (w = g^x) \wedge (0 < x)\}. \quad (33)$$

Params: $\mathbb{G} \xleftarrow{\$} GGen(\lambda), g \in_R \mathbb{G}$. Inputs: $w \in \mathbb{G}$. Witness: $x \in \mathbb{Z}$; Claim: $g^x = w \wedge 0 < x$.

1. $\mathcal{P}$ computes $x = \sum_{i=1}^4 a_i^2 \in \mathbb{Z}$ using the probabilistic Pollack-Treviño algorithm [49]. Let $P_i = g^{a_i^2}$.
2. $\mathcal{P}$ and $\mathcal{V}$ execute the PoKSE protocol for each P_i .
3. $\mathcal{V}$ accepts if all PoKSE's output is accept and at least one $P_i \neq 1$ and if $w = P_1 P_2 P_3 P_4$.

Fig. 9: The PoKPE protocol.

A.6 Aggregating Knowledge of Co-prime Roots (PoKCR)

In most of the applied NIZKs above, the verification equation checks a witness w_i as a random prime x_i th-root of an element a_i . These verification equations can be batched into a single check whenever $\forall i, j, i \neq j : \gcd(i, j) = 1$, i.e., there exists a constant-size proof for the following language,

$$\mathcal{R}_{\text{PoKCR}} = \{a \in \mathbb{G}^n; \mathbf{x} \in \mathbb{Z}^n : w = \phi(\mathbf{x}) \in \mathbb{G}\}, \quad (34)$$

where $\phi(\cdot) : \mathbb{Z}^n \rightarrow \mathbb{G}$ is a group homomorphism. The following protocol allows us to prove membership efficiently in the $\mathcal{R}_{\text{PoKCR}}$ language; the proof is a single group element. Note, the verifier has a slightly increased computation cost, as verifying the aggregated proof now requires $\mathcal{O}(n \log n)$ group operations with exponents of size $\max_i |x_i|$. Since, in the case of the Behemoth **PC** verifier, n is constant, this added computation overhead does not change the asymptotic overhead of our verifier.

Params: $\mathbb{G} \xleftarrow{\$} GGen(\lambda), g \in \mathbb{G}$. Inputs: $\mathbf{a} \in \mathbb{G}^n, \mathbf{x} \in \mathbb{Z}^n$ such that $\gcd(x_1, \dots, x_n) = 1$.
 Witness: $\mathbf{w} \in \mathbb{G}^n$ such that $w_i^{x_i} = a_i$.

1. $\mathcal{P}$ computes $w = \prod_{i=1}^n w_i$, and $\mathcal{P}$ sends w to $\mathcal{V}$.
2. $\mathcal{V}$ computes $x^* = \prod_{i=1}^n x_i$, and $y = \prod_{i=1}^n a_i^{x^*/x_i}$ using a recursive algorithm of Boneh et al. [16]. $\mathcal{V}$ accepts if $w^{x^*} = y$.

Fig. 10: The PoKCR protocol.